

Model Context Protocol (MCP)

Companion Guide v1.0

CIS Critical Security Controls v8.1

April 2026

Acknowledgments

The Center for Internet Security, Inc. (CIS®) would like to thank the many security experts who volunteer their time and talent to support the CIS Critical Security Controls® (CIS Controls®) and other CIS work. CIS products represent the effort of a veritable army of volunteers from across the industry, generously giving their time and talent in the name of a more secure online experience for everyone.

As a nonprofit organization driven by its volunteers, we are always in the process of looking for new topics and assistance in creating cybersecurity guidance. If you are interested in volunteering and/or have questions, comments, or have identified ways to improve this guide, please email us at: controlsinfo@cisecurity.org.

All references to tools or other products in this guide are provided for informational purposes only, and do not represent the endorsement by CIS of any particular company, product, or technology.

Principal Authors

Andrew Dannenberger, CIS
Shreyans Mehta, Cequence

Editors

Robin Regnier, CIS
Thomas Sager, CIS
Valecia Stocchetti, CIS

Contributors

Abhishek Iyer, Cybersecurity Leader
Christopher Misra, University of Massachusetts
Geoff Hancock, Founder-CISO, Cyber Bridge Solutions
Jack Zaldivar Jr., Databricks
Jonathan Sander, Astrix

This work is licensed under a Creative Commons Attribution-Non Commercial-No Derivatives 4.0 International Public License (the link can be found at <https://creativecommons.org/licenses/by-nc-nd/4.0/legalcode>).

To further clarify the Creative Commons license related to the CIS Controls® content, you are authorized to copy and redistribute the content as a framework for use by you, within your organization and outside of your organization for non-commercial purposes only, provided that (i) appropriate credit is given to CIS, and (ii) a link to the license is provided. Additionally, if you remix, transform, or build upon the CIS Controls, you may not distribute the modified materials. Users of the CIS Controls framework are also required to refer to (<http://www.cisecurity.org/controls/>) when referring to the CIS Controls in order to ensure that users are employing the most up-to-date guidance. Commercial use of the CIS Controls is subject to the prior approval of the Center for Internet Security, Inc. (CIS).

Contents

Executive Summary	1
Scope	2
Methodology	3
How to Use This Guide	5
Glossary	6
Control 1: Inventory and Control of Enterprise Assets	9
MCP Applicability	9
Safeguards	10
Additional MCP Considerations	11
Control 2: Inventory and Control of Software Assets	12
MCP Applicability	12
Safeguards	13
Additional MCP Considerations	14
Control 3: Data Protection	15
MCP Applicability	15
Safeguards	15
Additional MCP Considerations	18
Control 4: Secure Configuration of Enterprise Assets and Software	19
MCP Applicability	19
Safeguards	20
Additional MCP Considerations	22
Control 5: Account Management	23
MCP Applicability	23
Safeguards	23
Additional MCP Considerations	25
Control 6: Access Control Management	26
MCP Applicability	26
Safeguards	27
Additional MCP Considerations	29
Control 7: Continuous Vulnerability Management	30
MCP Applicability	30
Safeguards	30
Additional MCP Considerations	32
Control 8: Audit Log Management	33
MCP Applicability	33
Safeguards	33
Additional MCP Considerations	35

Control 9: Email and Web Browser Protections	36
MCP Applicability	36
Safeguards	36
Additional MCP Considerations	37
Control 10: Malware Defenses	38
MCP Applicability	38
Safeguards	38
Additional MCP Considerations	39
Control 11: Data Recovery	40
MCP Applicability	40
Safeguards	40
Additional MCP Considerations	41
Control 12: Network Infrastructure Management	42
MCP Applicability	42
Safeguards	42
Additional MCP Considerations	43
Control 13: Network Monitoring and Defense	44
MCP Applicability	44
Safeguards	44
Additional MCP Considerations	46
Control 14: Security Awareness and Skills Training	47
MCP Applicability	47
Safeguards	47
Additional MCP Considerations	49
Control 15: Service Provider Management	50
MCP Applicability	50
Safeguards	50
Additional MCP Considerations	51
Control 16: Application Software Security	52
MCP Applicability	52
Safeguards	52
Additional MCP Considerations	55
Control 17: Incident Response Management	56
MCP Applicability	56
Safeguards	56
Additional MCP Considerations	58
Control 18: Penetration Testing	59
MCP Applicability	59
Safeguards	59
Additional MCP Considerations	60
Conclusion	61

Appendix A: MCP Deployment Security Patterns	62
A.1: Local stdio Server Security	63
A.2: Remote Streamable HTTP Server Security	65
A.3: Gateway-Mediated Deployment Security	67
A.4: Third-Party MCP Server Security	69
A.5: MCP Apps Extension Security	70
A.6: MCP Authorization Extension Security	71
Appendix B: MCP CVEs Mapped to Best Practices	72
Appendix C: CIS Controls	73
Appendix D: Acronyms and Abbreviations	74
Appendix E: Links and Resources	75

Executive Summary

The Model Context Protocol (MCP) is an open standard designed to let artificial intelligence (AI) systems interact consistently with external tools, data sources, and services. Rather than relying on proprietary or model-specific integrations, MCP provides a common, interoperable framework so that different models, agents, and platforms can access the same set of capabilities in a controlled way. This increases modularity, improves auditability, and makes integration behavior (discovery, invocation, and logging) more predictable across models and platforms.

At its core, MCP defines how an AI model can request information, call tools, read structured documents, or interact with a system, without requiring bespoke, model-specific plugin implementations for each tool or data source. For enterprises operating in sensitive or complex environments, this creates a scalable and policy-aligned way to connect AI to internal systems while maintaining visibility and control over what the model can access.

A defining characteristic of MCP is its focus on explicit permissions, clear interface contracts, and auditable actions. Rather than broad or opaque access, each capability (whether retrieving data, running a command, or submitting a task) is granted individually.

More broadly, MCP helps standardize how AI agents operate in enterprise environments by abstracting away model-specific differences and providing a predictable communication layer. This supports consistent, safe integration across products, platforms, and vendors.

This guide provides practical, actionable guidance for applying CIS Controls v8.1 to systems that implement MCP. In CIS terms, MCP primarily expands the identity, access control, logging, and application security surfaces by formalizing how AI systems discover and invoke privileged capabilities. MCP introduces operational and security considerations that differ significantly from traditional integration models, requiring protections tailored to agent driven tool execution and context management. This guide interprets the CIS Controls in the context of MCP deployments and highlights additional considerations needed to protect these systems effectively.

Scope

This guide applies to enterprises deploying MCP systems that give AI applications controlled access to external tools, pre-written prompts, and data sources. The scope includes local Standard Input/Output (stdio) and remote Streamable Hypertext Transfer Protocol (HTTP) deployments across enterprise-controlled and third-party servers. This includes deployments where MCP enables state-changing operations (write actions) and access to sensitive enterprise resources, which require stronger authorization, auditing, and release controls. If third-party MCP servers are permitted, enterprises should treat them as software and service providers requiring onboarding, allowlisting, provenance validation, and continuous monitoring.

The guide addresses four MCP components – hosts, clients, servers, and gateways – each of which plays a distinct role in how AI applications discover, authorize, and invoke external capabilities. Note that while gateways are not part of the MCP specifications, they have emerged as a common part of real-world MCP infrastructure – especially where enterprises are looking to define secure operations. Component definitions appear in the [Glossary](#). Enterprises should inventory these components, assign ownership, and maintain an AI and MCP bill of materials for each deployment.

This guide does not cover general Large Language Model (LLM) security topics addressed in the [AI LLM Companion Guide](#). Enterprises using MCP within broader agent systems should also consult the [AI Agent Companion Guide](#) for guidance on multi-step planning, state management, and tool orchestration that operate above the MCP protocol layer. MCP controls do not replace model-layer controls (such as data protection, prompt/output handling, and model change management); they complement them.

Protocol Overview

Model Context Protocol (MCP) specifies how AI applications connect to MCP servers to discover and invoke capabilities in a consistent manner. The current MCP specification uses JavaScript Object Notation Remote Procedure Call (JSON-RPC) 2.0 for messaging across supported transports. Each JSON-RPC request includes an ID that correlates the request with its response, and request identifiers should be non-null and unique within a client session to ensure correlation and auditability. Common transports include stdio for local integrations and Streamable HTTP for network deployments.

MCP introduces a small set of primitives relevant to security and assessment:

- Tools are executable actions exposed by a server. Tool schemas define expected parameters, but servers and gateways must enforce allowlisting, least privilege, and server-side parameter validation. Never rely on the client or model to enforce tool authorization.
- Resources are retrievable data used as contextual input. Treat resource access as a data access path governed by enterprise classification, RBAC/ABAC, and audit logging. Retrieved content should be logged with resource identifiers and provenance.
- Prompts are reusable templates exposed by a server. Treat prompts as content-supply-chain inputs (not trusted instructions). Maintain prompt provenance, change control, and integrity protections, and log prompt identifiers/versions used in executions.

Methodology

This guide follows the structure and intent of CIS Controls v8.1 and is designed to be read alongside the primary CIS Controls document. It does not introduce new Controls or Safeguards. Instead, it interprets each Safeguard for MCP deployments by identifying the MCP components affected and the operational evidence that can demonstrate the Safeguard is working. This guide assumes authorization decisions are enforced deterministically by gateways and servers and are not delegated to model outputs.

For each Control (1 through 18), the guide provides:

- **MCP Applicability:** A short explanation of how the Control applies to MCP, including the main risk themes and why the Control matters.
- **Safeguards:** Guidance that adapts the CIS Safeguards to MCP-specific contexts. These are the primary actionable elements that demonstrate how MCP applies to the CIS Safeguards. Where appropriate, guidance is tied to MCP components (host, client, server, gateway), MCP primitives (tools, resources, prompts), and MCP transports (stdio and Streamable HTTP).
 - **Note:** “No Additional MCP Guidance” does not mean the Safeguard is irrelevant — it means the Safeguard as written already addresses MCP contexts without requiring additional guidance.
- **Additional MCP Considerations:** Additional nuances, edge cases, or contextual guidance that can help enterprises tailor Safeguard implementations to their particular AI architectures and risk profile.

This guide focuses specifically on securing the Model Context Protocol: transport, authorization, capability exposure, and tool execution boundaries. Additionally, the content in this guide relates to other CIS Controls companion guides including:

- MCP deployments inherit model-level security requirements from the [AI LLM Companion Guide](#)
- Agent-level controls such as orchestration, multi-step planning, and tool selection logic are addressed in the [AI Agent Companion Guide](#) and operate above the MCP protocol layer

Protocol-Layer Focus

This guide addresses MCP as an integration protocol rather than as a complete AI system. It covers how tools, resources, and prompts are discovered, authorized, and invoked – not how an agent decides which tools to use or how a model processes context. Single-step tool invocation security (allowlisting, authorization, validation, logging) is in scope even when agentic multi-step planning is out of scope. Enterprises building agentic systems should apply this guide alongside the [AI Agent Companion Guide](#).

Risk-Based Tailoring

Not all MCP deployments carry equal risk. Apply guidance proportionally based on:

- Deployment pattern
- Tool sensitivity, which should be categorized at a minimum as: (1) read-only non-sensitive, (2) read-only sensitive, (3) write/change state, or (4) irreversible/high impact (external communications, financial/legal commitments, security posture changes)
- Data classification of accessible resources
- Whether third-party servers are permitted

Deployment Pattern Categories

This guide uses four deployment pattern categories to provide conditional guidance. See [Appendix A: MCP Deployment Security Patterns](#) for comprehensive security guidance specific to each pattern:

- Local stdio servers running on endpoints or developer workstations
- Remote Streamable HTTP servers accessed directly
- Gateway-mediated deployments that centralize policy enforcement
- Third-party MCP servers from registries or external providers

For high-impact workflows, gateway-mediated deployments are the recommended default to centralize identity binding, policy enforcement, logging, and kill-switch controls. All technical guidance is grounded in the current MCP specification and supporting security guidance referenced in [Appendix E: Links and Resources](#).

How to Use This Guide

[Implementation Groups \(IG1, IG2, IG3\)](#) continue to guide prioritization as with all preceding CIS Controls guides. Enterprises deploying MCP in high-impact workflows may require Safeguards from higher Implementation Groups regardless of their overall IG classification.

This guide assumes the enterprise has implemented the CIS Controls appropriate to its operating environment. Each section corresponds to a Control, first summarizing key MCP security risk areas, then providing Safeguard-level applicability to MCP components, primitives, and transports.

Some Safeguards reference detection, monitoring, or validation capabilities that may require custom development or emerging tooling. While some commercial security products are beginning to include MCP-specific signatures, detections, and compliance checks, enterprises should still plan for custom implementations or rely on compensating controls until the broader tooling ecosystem matures. Until tooling matures, enterprises should prioritize gateway-mediated deployments, strict allowlisting of MCP servers, compensating controls through endpoint detection and response (EDR) and application allowlisting on endpoints, and manual review processes for capability changes. This guide provides control-level guidance for implementation and assessment purposes. Enterprises should develop operational procedures, detection rules, and incident response playbooks based on their specific deployment patterns and risk tolerance.

Transport and Authorization Assumptions

For Streamable HTTP deployments, this guide assumes enterprises use a centralized enterprise authorization approach aligned with the MCP authorization model. MCP servers should delegate user authentication to the enterprise identity provider (IdP) via OAuth 2.1 rather than directly collecting or handling user credentials. Restrict token exposure to the minimum set of components required to enforce access, and apply least privilege through scoped access and explicit approval of server capabilities. Tokens should be scoped to specific servers and capabilities (audience-restricted), short-lived where feasible, and never logged in plaintext. Gateways and servers should validate token audience, issuer, and expiry on every request.

For stdio deployments, security rests on local process boundaries, operating system identities, and file system permissions. Treat local MCP servers as controlled software assets: run them under least-privilege accounts, restrict environment variable scope, control filesystem roots, and apply application allowlisting to approved server binaries. Downstream network calls from stdio servers to backend APIs are subject to the same transport security requirements as Streamable HTTP deployments.

Important: a model or client request is not to be treated as an authorization decision. Authorization and policy enforcement must be implemented deterministically at the gateway and/or server layer, independent of model output.

Glossary

Allowlist	A security control that permits only explicitly approved items such as servers, tools, or network destinations. MCP deployments use allowlists to restrict which servers may be installed, which tools may be invoked, and which endpoints servers may contact.
Authorization Server	The OAuth component that authenticates users, issues access tokens, and manages consent. MCP clients obtain tokens from the authorization server to access protected MCP servers. Must support PKCE with `S256` and should implement Protected Resource Metadata discovery.
Capability Negotiation	The initialization process where clients and servers exchange supported capabilities to establish session features.
Client	The protocol component within a host that establishes connections to MCP servers, handles capability negotiation, and routes messages between the host and servers.
Client ID Metadata Document (CIMD)	A JSON document hosted at an HTTPS URL controlled by the client that describes OAuth client metadata used for validation and registration. In MCP, CIMD is the preferred alternative to Dynamic Client Registration.
Confused Deputy	An attack where a trusted component with elevated privileges is manipulated into performing unauthorized actions on behalf of an attacker. In MCP, this commonly involves proxy servers that use static client IDs for third-party APIs, allowing the proxy to authorize third-party API access using its own credentials without per-user consent verification.
DNS Rebinding	An attack where an attacker-controlled domain initially resolves to an external IP, then re-resolves to `127.0.0.1`, allowing remote web content to send requests to localhost-bound servers.
Elicitation	MCP capability for server-initiated, structured user input via `elicitation/create`. Risks include credential harvesting, deceptive prompts, and consent manipulation. Mitigate with least-privilege access, auditable logging, and user training; apply the same controls as Sampling.
Gateway	An optional intermediary that mediates access to multiple MCP servers, providing centralized authorization, logging, rate limiting, and policy enforcement.
Host	The user-facing AI application that manages MCP client connections, orchestrates LLM interactions, and enforces access controls. Examples include Claude Desktop, Visual Studio Code (VS Code), and custom AI applications.
Human-in-the-Loop (HITL)	A control pattern requiring explicit human approval before executing tool invocations, particularly for state-changing, high-risk, or sensitive operations. The MCP specification states that a human should always have the ability to deny tool invocations.
Implementation Group (IG)	Grouped IG1, IG2, and IG3, these are a way for enterprises to prioritize the implementation of the CIS Controls.
Initialization Handshake	The protocol exchange where a client sends an `initialize` request declaring its capabilities and protocol version, and the server responds with its own capabilities, followed by the client sending an `initialized` notification. This handshake must be completed before any other protocol messages are exchanged.
JSON-RPC	JSON Remote Procedure Call, the messaging protocol used by MCP for all client-server communication. MCP uses JSON-RPC 2.0 with requests containing method names, parameters, and unique identifiers that correlate requests with responses.
Kill Switch	A control or mechanism that allows rapid disabling of a model, endpoint, tool capability, agent, or entire AI subsystem in response to an incident.
listChanged Notification	A notification sent when available tools, resources, or prompts change, triggering capability refresh.
MCP Apps	An extension to MCP that allows an MCP server to provide interactive user interface content rendered by the host application. MCP Apps introduce a browser execution context into the trust boundary, requiring isolation controls such as sandboxed rendering, restrictive content policies, and explicit user approval for permission requests. See Appendix A.5: MCP Apps Extension Security .
MCP-Session-Id	An HTTP header in Streamable HTTP that maintains session state across requests.

Model Context Protocol (MCP)	An open standard protocol for connecting AI applications to external tools, data sources, and services through a client-server architecture using JSON-RPC 2.0 messaging.
OAuth	An open standard authorization framework that lets a client application obtain limited access to protected resources without receiving the user's credentials. OAuth defines roles, flows, and token mechanisms that enable secure delegated authorization. MCP's authorization model aligns with OAuth 2.1 patterns, including mandatory PKCE with S256.
Origin Validation	A security control where servers verify the Origin header on incoming HTTP requests to prevent DNS rebinding attacks. MCP servers must reject requests with unexpected Origin values and return `HTTP 403` for invalid Origins.
PKCE (Proof Key for Code Exchange)	An OAuth extension that prevents authorization code interception attacks. MCP requires PKCE with the `S256` challenge method for all OAuth flows.
Primitive	A core capability type that MCP clients and servers can expose to each other. Server-side primitives include tools, resources, and prompts. Clients can also expose a sampling primitive, which allows servers to request language model completions.
Prompt	The input content provided to a model or agent, including instructions, questions, or data examples. <i>In the MCP context, it also refers to a reusable template exposed by an MCP server that provides preconfigured instructions or workflows; MCP prompts are user-controlled and selected explicitly by the user.</i>
Prompt Injection	An attack where malicious instructions are embedded in content processed by an LLM to manipulate its behavior. Indirect prompt injection occurs when hostile instructions arrive through retrieved resources, tool outputs, or other external content rather than direct user input.
Protected Resource Metadata	A mechanism for servers to advertise OAuth requirements at a well-known endpoint, including authorization server location and supported scopes.
Resource	Data exposed by an MCP server, identified by a Uniform Resource Identifier (URI). Resources are application-controlled, meaning the host application decides how and when to use them.
Roots	Filesystem boundaries defined by clients using `file://` URIs that specify which directories servers may access.
Rug Pull Attack	Updating a previously benign MCP server to include malicious functionality after users have approved it.
Sampling	A capability allowing servers to request LLM completions through the client via `sampling/createMessage.` Enables server-side agent loops while keeping model access under client control.
Scope	An OAuth mechanism that limits the permissions granted by an access token. MCP servers declare required scopes, and authorization servers issue tokens restricted to approved scopes. Clients should request minimal scopes necessary for intended operations.
Server	A service that exposes tools, resources, and prompts to MCP clients. Servers run locally as subprocesses using stdio or remotely over HTTP using Streamable HTTP.
Server-Sent Events (SSE) Transport, Deprecated	Older HTTP transport using separate endpoints for SSE and POST. Replaced by Streamable HTTP.
stdio	Standard Input/Output is a local transport where the client launches the server as a subprocess and communicates via standard input and output using newline-delimited JSON-RPC.
Streamable HTTP	HTTP-based transport for remote servers using a single endpoint for POST requests from client to server and optional GET requests for Server-Sent Events streaming.
Tasks	An MCP primitive for long-running operations where a request returns a task handle. Clients can query task status and retrieve results asynchronously. Tasks can include states such as working, `input_required`, `completed`, `failed`, and `cancelled`.
Third-party Server	An externally sourced MCP server from community registries, open-source repositories, or commercial vendors. Such servers require additional vetting, including provenance verification, capability review, and ongoing monitoring for rug pull attacks.
TLS (Transport Layer Security)	The cryptographic protocol that provides encrypted communication. MCP requires TLS 1.2 or later for all Streamable HTTP connections to protect data in transit between clients, servers, and authorization servers.

Token Audience Binding	Binding OAuth tokens to specific servers using the resource parameter, preventing token replay across different servers.
Token Passthrough	Forwarding a token received from an MCP client to a downstream API. MCP forbids token passthrough; servers that call backend or downstream APIs should obtain separate tokens.
Tool	An external capability that a model or agent invokes, such as APIs, databases, execution engines, browsers, file handlers, or custom actions. In the MCP context, tools are callable operations exposed by an MCP server. Tools are model-controlled, meaning the LLM selects which tools to invoke based on their descriptions.
Tool Poisoning	Embedding malicious instructions in tool names, descriptions, or parameter schemas to manipulate LLM behavior.
Tool Wrapper	A software component that adapts an external API, service, or system into an MCP-compatible tool. Tool wrappers translate between the MCP tool interface (JSON-RPC requests with declared schemas) and the underlying system's native interface. They may be internally developed or obtained from registries and package ecosystems.
Typosquatting	A supply chain attack technique that uses package or server names closely resembling legitimate ones to trick users into installing malicious components. In MCP environments, typosquatting targets server registries and package ecosystems.

Control 1: Inventory and Control of Enterprise Assets

Actively manage (inventory, track, and correct) all enterprise assets (end-user devices, including portable and mobile; network devices; non-computing/Internet of Things (IoT) devices; and servers) connected to the infrastructure physically, virtually, remotely, and those within cloud environments, to accurately know the totality of assets that need to be monitored and protected within the enterprise. This will also support identifying unauthorized and unmanaged assets to remove or remediate.

MCP Applicability

MCP expands the asset inventory beyond traditional endpoints to include MCP hosts, clients, servers, gateways, declared tools, resources, prompt endpoints, internal and external MCP registries, and supporting identity and authorization components. If these MCP components are not formally inventoried, they can become invisible dependencies that create shadow pathways into business systems. For MCP, the effective “asset” is the capability surface (tools, resources, prompts), the identities used to access them, and the enforcement points that authorize and log execution.

MCP-related assets can be harder to inventory using traditional methods. Challenges include:

- Teams often add servers for evaluation or development and then abandon them, creating short-lived installs.
- stdio servers may run as child processes under user or developer privileges and may not appear as managed services. Inventory must capture process lineage, execution user, and the local files and secrets accessible to that process context.
- Inventory must capture declared tools, resources, and prompts, as well as logging configuration, since these elements define effective access and represent the system’s capability exposure.
- Registries function as discovery endpoints that abstract underlying infrastructure. Inventory must capture not only the registry URL but also the specific server versions and metadata (publisher, SHA-256 hashes) it authorizes for the enterprise.
- MCP servers can change declared capabilities over time (new tools, resources, prompts) without obvious infrastructure changes. Inventory must detect capability drift, not just new hosts.

Safeguards

CIS Control 1: Inventory and Control of Enterprise Assets

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
1.1	Establish and Maintain Detailed Enterprise Asset Inventory	Establish and maintain an accurate, detailed, and up-to-date inventory of all enterprise assets with the potential to store or process data, to include: end-user devices (including portable and mobile), network devices, non-computing/IoT devices, and servers. Ensure the inventory records the network address (if static), hardware address, machine name, enterprise asset owner, department for each asset, and whether the asset has been approved to connect to the network. For mobile end-user devices, MDM type tools can support this process, where appropriate. This inventory includes assets connected to the infrastructure physically, virtually, remotely, and those within cloud environments. Additionally, it includes assets that are regularly connected to the enterprise's network infrastructure, even if they are not under control of the enterprise. Review and update the inventory of all enterprise assets bi-annually, or more frequently.	●	●	●	Maintain an enterprise inventory of MCP components and a server registry as the authoritative record. Establish the registry and keep it current through routine reviews and updates when changes occur. Track all hosts, clients, servers, and gateways. Include named owner, escalation contact, and decommission planning for each component. For each component, record the deployment type, declared capabilities, authorization details, transport and supported protocol versions, and version. For third-party servers, record the source and approval status. For each server, also record: risk tier (read-only vs. write vs. irreversible), containment lever owner (who can disable tools, server, gateway), and capability baseline snapshot (hash or versioned export of tools, resources, prompts).
1.2	Address Unauthorized Assets	Ensure that a process exists to address unauthorized assets on a weekly basis. The enterprise may choose to remove the asset from the network, deny the asset from connecting remotely to the network, or quarantine the asset.	●	●	●	Detect and remove or quarantine unauthorized MCP servers, clients, and gateways, and treat unauthorized capability expansions (new tools, resources, prompts on a known server) as unauthorized assets requiring immediate review and re-approval before production use.
1.3	Utilize an Active Discovery Tool	Utilize an active discovery tool to identify assets connected to the enterprise's network. Configure the active discovery tool to execute daily, or more frequently.		●	●	Use active discovery to identify stdio MCP server executables, JSON configuration files, and running processes across endpoints. During initialization, capture configured capabilities such as tool schemas and resource URI patterns for security review. Cross-reference actively discovered servers against the approved enterprise registry. Flag unknown MCP server binaries and configurations, unexpected execution paths, and any server found in the environment that lacks a corresponding, integrity-verified entry in the registry including any capability declarations that do not match the approved registry baseline.
1.4	Use Dynamic Host Configuration Protocol (DHCP) Logging to Update Enterprise Asset Inventory	Use DHCP logging on all DHCP servers or Internet Protocol (IP) address management tools to update the enterprise's asset inventory. Review and use logs to update the enterprise's asset inventory weekly, or more frequently.		●	●	DHCP logging is not a primary inventory signal for MCP stdio deployments; for network-based deployments, use gateway logs, service discovery inventories, and passive network monitoring to identify MCP endpoints.

CIS Control 1: Inventory and Control of Enterprise Assets

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
1.5	Use a Passive Asset Discovery Tool	Use a passive discovery tool to identify assets connected to the enterprise's network. Review and use scans to update the enterprise's asset inventory at least weekly, or more frequently.				Passive network analysis identifies MCP communications across enterprise segments. Focus on session initialization and capability negotiation messages to confirm exposed server features. Monitor authentication and authorization events associated with MCP access. Alert on unknown or unauthorized MCP endpoints and on unusual volumes or sequences of tool invocation. Alert on anomalous tool invocation sequences (new tool names, unusual call order, high-frequency bursts), and on authentication failures that indicate enumeration or replay.

Additional MCP Considerations

- Use the Enterprise MCP Registry as the authoritative, versioned server and tool catalog. This registry must maintain a complete change history of tool schemas, resource URI patterns, and publisher metadata so that investigators can confirm the “known-good” authorized state at any point in time.
- When MCP hosts support dynamic server discovery or installation, restrict who can add servers to the approved enterprise registry. Log first-seen server registrations and connections, and alert on any production connections to servers not found in the registry.
- Treat `listChanged` notifications as inventory signals. Unexpected changes to a server's declared tools, resources, or prompts must be cross-referenced against the approved registry entry. Deviations should trigger investigation and re-validation against the authorized baseline.
- Default policy should be deny-by-default for newly declared capabilities until re-approved, especially in production.
- Use the `clientInfo` and `serverInfo` fields exchanged during the initialization handshake as inventory data sources. These fields include name and version, and may also include `description` and `websiteUrl`. Capture them at the gateway or host to automatically populate and validate the MCP asset inventory.
- Treat `clientInfo` and `serverInfo` as self-asserted metadata and validate against allowlisted identities, signed artifacts, and approved registries.
- Leverage the functional metadata within the registry to perform static inventory validation. If a server's capability declaration (tools, resources, prompts) during initialization differs from its registered profile, treat the event as unauthorized asset expansion or a potential “Rug Pull” attack.
- Explicitly inventory the use of the Official MCP Registry (registry.modelcontextprotocol.io). Implement a “snapshot” or “mirror” policy to ensure that updates to external registries do not automatically modify the enterprise's local inventory or trust boundaries without manual review.

Control 2: Inventory and Control of Software Assets

Actively manage (inventory, track, and correct) all software (operating systems and applications) on the network so that only authorized software is installed and can execute, and that unauthorized and unmanaged software is found and prevented from installation or execution.

MCP Applicability

With MCP in place, software inventory becomes even more critical due to its network of software components. MCP servers expose capabilities, MCP clients mediate communication, and AI hosts rely on software interfaces to trigger actions or retrieve data. Since MCP encourages modular integration between AI systems and other tools, enterprises may see a rapid increase in the number of small, specialized services. Since MCP components can bridge into sensitive systems, treat MCP servers and gateways as integration middleware requiring higher change control, patch SLAs, and code provenance standards than typical internal services. Gateways and policy enforcement components are security-critical software assets and must be inventoried and governed as such.

Software inventory for MCP can be hard to manage. Challenges include:

- Specification and Software Development Kit (SDK) updates can change transport and authorization behavior across environments, leading to version drift and compatibility issues.
- Servers and tool wrappers may come from registries or internal builds with mixed provenance signals, introducing supply-chain variability.
- MCP servers often pull in numerous libraries for authorization, message parsing, and integrations, which increases patch scope and the attack surface associated with transitive dependencies.
- Tool wrappers often embed credentials or rely on environment secrets, making software inventory inseparable from secrets management and runtime identity controls.
- Local studio servers can be installed via developer workflows and bypass normal enterprise deployment pipelines, increasing the risk of unreviewed code execution on endpoints.

Safeguards

CIS Control 2: Inventory and Control of Software Assets

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
2.1	Establish and Maintain a Software Inventory	Establish and maintain a detailed inventory of all licensed software installed on enterprise assets. The software inventory must document the title, publisher, initial install/use date, and business purpose for each entry; where appropriate, include the Uniform Resource Locator (URL), app store(s), version(s), deployment mechanism, decommission date, and number of licenses. Review and update the software inventory bi-annually, or more frequently.	●	●	●	Maintain an inventory of MCP software assets: server executables, client libraries and SDKs, gateway components, OAuth libraries, and JSON-RPC frameworks. For each component, record package name and version, source registry or repository, integrity data such as hashes or signatures, and the MCP specification version supported. Include third-party servers with source and approval status. For deployed instances, record transport type, declared capabilities, and authorization configuration. Link deployment-specific capability and configuration details to records maintained under Controls 1 and 4. Maintain an SBOM/dependency manifest for MCP components, including transitive dependencies, to accelerate vulnerability response. Verify integrity at intake and again at provisioning and deployment to detect tampering.
2.2	Ensure Authorized Software is Currently Supported	Ensure that only currently supported software is designated as authorized in the software inventory for enterprise assets. If software is unsupported, yet necessary for the fulfillment of the enterprise's mission, document an exception detailing mitigating controls and residual risk acceptance. For any unsupported software without an exception documentation, designate as unauthorized. Review the software list to verify software support at least monthly, or more frequently.	●	●	●	Use only supported MCP software. Verify active maintenance, vulnerability handling, and update cadence for servers, SDKs, and gateways. Migrate unsupported components to supported alternatives. Track protocol and transport support as part of life cycle management and ensure authorization features required by your deployment pattern are supported and tested. Define an enterprise-supported baseline for MCP spec versions and SDK versions and block production deployments outside this baseline unless an exception is formally approved with documented compensating controls and a defined expiry date.
2.3	Address Unauthorized Software	Ensure that unauthorized software is either removed from use on enterprise assets or receives a documented exception. Review monthly, or more frequently.	●	●	●	Identify and remove unauthorized MCP servers, clients, and tool wrappers that have not undergone security review or are not in the approved software inventory. For stdio deployments, default-deny unauthorized binaries via application allowlisting and quarantine processes rather than relying solely on manual detection.
2.4	Utilize Automated Software Inventory Tools	Utilize software inventory tools, when possible, throughout the enterprise to automate the discovery and documentation of installed software.		●	●	Use automated discovery to identify MCP server files, JSON configuration files that declare servers and capabilities, and active MCP processes. Scan common installation paths and package manager records such as `npm` for TypeScript and `pip` for Python. Parse process command lines that indicate MCP server execution. For Streamable HTTP deployments, detect JSON-RPC traffic patterns to MCP endpoints and associated streaming responses where applicable, and correlate observed servers to the approved MCP registry.

CIS Control 2: Inventory and Control of Software Assets

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
2.5	Allowlist Authorized Software	Use technical controls, such as application allowlisting, to ensure that only authorized software can execute or be accessed. Reassess bi-annually, or more frequently.		●	●	Enforce an allowlist for MCP servers and tool wrappers using versioned entries with integrity verification, deny-by-default per Control 1. Install only from enterprise approved registries or vetted artifact repositories, block direct URL, personal registry, and local source installs in production, and verify signatures or hashes at intake and again at provisioning. Allowlisting must cover both the server artifact and the approved capability set; capability expansions are blocked until reviewed and promoted.
2.6	Allowlist Authorized Libraries	Use technical controls to ensure that only authorized software libraries, such as specific .dll, .ocx, and .so files, are allowed to load into a system process. Block unauthorized libraries from loading into a system process. Reassess bi-annually, or more frequently.		●	●	Allowlist software libraries and package dependencies used by MCP components, including JSON-RPC frameworks, OAuth libraries, transport libraries, SDKs, and any native modules loaded into MCP server or client processes. Prefer reproducible builds and continuous integration (CI) enforcement that blocks unapproved dependency changes before artifacts are produced.
2.7	Allowlist Authorized Scripts	Use technical controls, such as digital signatures and version control, to ensure that only authorized scripts, such as specific .ps1 and .py files, are allowed to execute. Block unauthorized scripts from executing. Reassess bi-annually, or more frequently.			●	For MCP servers implemented in Python, JavaScript, TypeScript, or shell, enforce strict script execution controls by allowing only pre-approved scripts and dependencies. Verify integrity with cryptographic hashing or code signing. Prefer reproducible builds and CI enforcement that blocks unapproved dependency changes before artifacts are produced.

Additional MCP Considerations

- Track server and tool wrapper provenance, including package source, commit or artifact identifiers, and whether the component is internally built or externally sourced.
- Maintain a policy for supported SDK and specification versions, including how upgrades are tested and how deprecated transports or features are handled.
- Ensure SBOMs are accessible to incident response (IR) and vulnerability management teams to support rapid triage when MCP-related CVEs emerge.

Control 3: Data Protection

Develop processes and technical controls to identify, classify, securely handle, retain, and dispose of data.

MCP Applicability

As AI-driven workflows expand, data paths multiply and exposure risk grows. MCP introduces structured communication between AI applications and the systems they use, so weaknesses in classification, transmission, or logging directly affect an enterprise’s data protection posture.

MCP creates data flows across tool inputs and outputs, resource retrievals, server-initiated sampling requests `sampling/createMessage`, prompt templates, and operational logs. It is important to apply classification and handling rules to these flows, not only to the systems of record. Treat MCP prompts, tool inputs/outputs, sampling artifacts, and operational logs as data assets subject to the same classification, retention, and access controls as systems of record. Treat them as part of the enterprise’s data landscape and protect the full data life cycle across MCP: ingestion → retrieval → context assembly → tool execution → logging/caching → retention → disposal.

Key data protection considerations in MCP environments:

- Classify and handle MCP inputs, outputs, and logs based on the most sensitive data they may contain, including aggregated context.
- Control what enters model context from tools and resources, especially untrusted or externally sourced content.
- Reduce data exposure by minimizing retained context, redacting sensitive fields in logs, and restricting access to stored artifacts.
- Define and enforce redaction/tokenization rules for tool outputs and resource content before they enter model context or logs.

Safeguards

CIS Control 3: Data Protection

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
3.1	Establish and Maintain a Data Management Process	Establish and maintain a documented data management process. In the process, address data sensitivity, data owner, handling of data, data retention limits, and disposal requirements, based on sensitivity and retention standards for the enterprise. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	Apply the enterprise data management process to MCP data, including tool inputs and outputs, resource payloads, prompts, logs, session data, caches, and sampling artifacts, based on sensitivity and retention requirements. Assign data owners for MCP-generated artifacts (prompt templates, context stores, caches, sampling records) and record where each artifact class is stored and audited.
3.2	Establish and Maintain a Data Inventory	Establish and maintain a data inventory based on the enterprise’s data management process. Inventory sensitive data, at a minimum. Review and update inventory annually, at a minimum, with a priority on sensitive data.	●	●	●	Inventory all MCP-reachable data sources and data categories. Record the backend system for each tool and the source and URI patterns for each resource. Record resource subscription configurations, including which resources support change notifications and which components receive updates. Include MCP-created data stores such as context caches, task state, sampling logs, and any persisted tool outputs. Record which components receive resource subscription updates and whether each subscription is approved for the data classification involved.

CIS Control 3: Data Protection

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
3.3	Configure Data Access Control Lists	Configure data access control lists based on a user's need to know. Apply data access control lists, also known as access permissions, to local and remote file systems, databases, and applications.	●	●	●	Enforce access control server-side (and/or at the gateway) for each tool and resource using an ACL scoped by resource, action, and authenticated identity. For Streamable HTTP, validate identity and claims, including scopes and audience, and enforce the ACL so that only permitted principals can act on protected resources. For stdio, bind tool and resource access to the OS identity and restrict file roots and environment access by policy.
3.4	Enforce Data Retention	Retain data according to the enterprise's documented data management process. Data retention must include both minimum and maximum timelines.	●	●	●	Set minimum and maximum retention periods for MCP data caches, including tool outputs, resource content, context stores, and sampling data, aligned to the classification of data flowing through each server. Retention must be tiered by tool sensitivity and data classification, and enforced through automated deletion with audit evidence.
3.5	Securely Dispose of Data	Securely dispose of data as outlined in the enterprise's documented data management process. Ensure the disposal process and method are commensurate with the data sensitivity.	●	●	●	Securely delete server caches, sensitive logs, and temporary files. Clear OAuth tokens stored by HTTP servers. Remove session data such as 'MCP-Session-Id' mappings. Ensure cleanup on normal shutdown and on failure paths for both stdio and Streamable HTTP deployments. Clear refresh tokens and any stored client credentials where applicable. Use crash-safe deletion patterns or envelope encryption with key shredding for sensitive caches.
3.6	Encrypt Data on End-User Devices	Encrypt data on end-user devices containing sensitive data. Example implementations can include: Windows BitLocker®, Apple FileVault®, Linux® dm-crypt.	●	●	●	Use full-disk encryption on endpoints that run stdio servers which may access sensitive data or cache resources. stdio servers run with user privileges and can read files and environment variables. Define allowed data classes for (a) tool parameters, (b) tool outputs, (c) resource payloads, and (d) prompt/context injection, and enforce these rules at the server or gateway boundary. NOTE: These Safeguards represent "Compensating Controls" for local stdio deployments where protocol-level network isolation is not possible.
3.7	Establish and Maintain a Data Classification Scheme	Establish and maintain an overall data classification scheme for the enterprise. Enterprises may use labels, such as "Sensitive," "Confidential," and "Public," and classify their data according to those labels. Review and update the classification scheme annually, or when significant enterprise changes occur that could impact this Safeguard.		●	●	Classify data accessed through MCP tools and resources, inheriting the classification of underlying databases, file systems, and APIs. Define what data may appear in tool parameters, outputs, and resource payloads by classification level, and enforce these rules at the server or gateway boundary.

CIS Control 3: Data Protection

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
3.8	Document Data Flows	Document data flows. Data flow documentation includes service provider data flows and should be based on the enterprise's data management process. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.		●	●	Document end-to-end data flows from tool invocations and resource retrievals through back-end systems into model context. Map servers to their data sources and identify tools that can access sensitive data, noting where content enters prompts. Flag high-risk sources and record resource annotations, including MCP metadata such as audience and priority that indicate intended consumers and handling precedence. Capture transfers outside the enterprise and align retention and deletion with policy. Document where content enters prompts and context and which controls enforce classification, redaction, and retention at each hop. Treat metadata (e.g., audience and priority) as handling signals, not as authorization; enforce access with identity and policy.
3.9	Encrypt Data on Removable Media	Encrypt data on removable media.		●	●	No Additional MCP Guidance
3.10	Encrypt Sensitive Data in Transit	Encrypt sensitive data in transit. Example implementations can include: Transport Layer Security (TLS) and Open Secure Shell (OpenSSH).		●	●	Require TLS 1.2 or later for Streamable HTTP. Validate certificates and reject weak ciphers and downgrade attempts. For stdio, protect data via OS process isolation, least-privilege execution, and endpoint encryption; treat any subsequent network calls from the server to backend APIs as in-transit data requiring TLS. Ensure back-end API calls from servers also use TLS. Gateways must terminate TLS with proper certificate management and can use mutual TLS for higher security. For high-risk deployments, use mutual TLS between gateway and server and enforce strict certificate validation.
3.11	Encrypt Sensitive Data at Rest	Encrypt sensitive data at rest on servers, applications, and databases. Storage-layer encryption, also known as server-side encryption, meets the minimum requirement of this Safeguard. Additional encryption methods may include application-layer encryption, also known as client-side encryption, where access to the data storage device(s) does not permit access to the plain-text data.		●	●	Encrypt sensitive MCP data at rest. This includes server caches, vector stores, file systems exposed by tools and any stored OAuth tokens or session data for HTTP servers. Protect configuration files that may contain authorization metadata, OAuth client credentials, or capability declarations. Store OAuth client secrets and API keys in a secrets manager, not in plaintext configuration files, and restrict filesystem permissions on capability and configuration declarations.
3.12	Segment Data Processing and Storage Based on Sensitivity	Segment data processing and storage based on the sensitivity of the data. Do not process sensitive data on enterprise assets intended for lower sensitivity data.		●	●	Isolate servers that access highly sensitive data. Separate third-party servers from enterprise-developed servers. Limit sampling to approved model endpoints and match provider contracts to data handling needs. Consider dedicated gateways with tighter authorization for high-risk groups.

CIS Control 3: Data Protection

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
3.13	Deploy a Data Loss Prevention Solution	Implement an automated tool, such as a host-based Data Loss Prevention (DLP) tool to identify all sensitive data stored, processed, or transmitted through enterprise assets, including those located onsite or at a remote service provider, and update the enterprise's data inventory.				Where DLP tooling is available or can be extended to inspect MCP traffic, apply DLP controls to tool responses and resource retrievals that enter model context. Monitor sampling requests for exposure risk. Filter outputs to prevent leakage. For Streamable HTTP deployments, gateway-based inspection is the most practical enforcement point. stdio deployments may require host-level controls or manual review until MCP-aware DLP tooling matures. Default enforcement for Streamable HTTP should be at the gateway; for stdio deployments, enforce via endpoint DLP/EDR policies and restrict which servers may run locally.
3.14	Log Sensitive Data Access	Log sensitive data access, including modification and disposal.				Log tool and resource access with enough metadata for audit while avoiding sensitive content. Record tool or resource identifiers, identity, server, session, timestamp and request IDs. Capture full parameters only under defined workflows and protect those records accordingly. Use two-tier logging: metadata logs by default, and tightly controlled full-content capture only for approved investigations with elevated access, short retention, and audit trails.

Additional MCP Considerations

- All MCP tool definitions, resources, and prompts contained in MCP client files, MCP registry definitions, or anywhere else must not contain any sensitive information. A common bad practice is for tool definitions to contain secrets (e.g., passwords, tokens, keys) or prompts and resources to contain proprietary information. Avoid this at all costs.
- Apply data minimization to context construction. Limit returned fields, truncate long records and avoid full document ingestion when excerpts meet the need.
- If sampling is enabled, restrict which servers may use it and limit sampling to approved model endpoints. Treat sampling requests as outbound transfers and apply the same boundary and data handling controls as other cross-boundary flows.
- Inspect sampling request metadata for abuse. Watch for unusually large prompts, repeated near-duplicate requests and requests from higher-risk servers. Set rate limits per server and per session and require elevated approval for requests that exceed thresholds.
- Apply structured output validation and redaction to tool outputs before injecting into model context to prevent accidental leakage or instruction injection.

Control 4: Secure Configuration of Enterprise Assets and Software

Establish and maintain the secure configuration of enterprise assets (end-user devices, including portable and mobile; network devices; non-computing/IoT devices; and servers) and software (operating systems and applications).

MCP Applicability

MCP introduces new architectural layers, such as hosts, servers, gateways, and identity systems, where configuration weaknesses can lead to significant security issues. Since these components work together, a single misconfiguration can ripple across the ecosystem, potentially compromising entire AI-driven workflows, tool chains, or backend interactions.

Across these layers, key considerations include transport exposure, authorization behavior, and capability configuration, each of which influences how safely the system operates. When these elements are not configured correctly, MCP can unintentionally expose powerful tool paths or reveal sensitive data through a single interface. The MCP Registry is meant to serve as the central repository for these configuration baselines. It should define the “intended state” of tool schemas, environment variable requirements, and resource URI patterns, allowing for automated detection of configuration drift or unauthorized capability expansion.

Key secure configuration considerations in MCP environments include:

- stdio servers require clear process boundaries and correct message handling. For Streamable HTTP deployments, bind services to the minimum necessary interfaces, enforce TLS, restrict exposure behind gateways and reverse proxies, and validate request origin where applicable to reduce transport exposure.
- Enforce consistent Open Authorization (OAuth) configuration across servers and gateways, including discovery, Proof Key for Code Exchange (PKCE), scope design, and token handling, to prevent privilege expansion and ensure authorization integrity.
- Include tools, resources, and prompts in the configuration baseline, with centrally managed approved server lists, tool allowlists, logging rules, origin checks, and session handling.
- Maintain versioned baselines and check for drift continuously and after each change.
- Treat MCP capability configuration (tools, resources, prompts), authorization settings, and gateway policy as configuration-as-code with version control, review, and promotion pipelines.

Safeguards

CIS Control 4: Secure Configuration of Enterprise Assets and Software

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
4.1	Establish and Maintain a Secure Configuration Process	Establish and maintain a documented secure configuration process for enterprise assets (end-user devices, including portable and mobile, non-computing/IoT devices, and servers) and software (operating systems and applications). Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	Create versioned secure configuration baselines for MCP servers, clients, and gateways. For stdio, restrict file access to declared roots, minimize process privileges, and ensure standard output (stdout) is reserved for protocol messages and standard error (stderr) for diagnostics, and prevent sensitive data from being written to either stream without redaction controls. This process should utilize the Enterprise MCP Registry to define the authorized schema and functional baseline for every approved server. For Streamable HTTP, enforce OAuth with PKCE, validate the Origin header, bind local-only servers to '127.0.0.1', and treat 'MCP-Session-Id' as a session state rather than an authorization mechanism. Do not treat session identifiers (MCP-Session-Id) as identity or authorization claims. See Appendix A for transport-specific hardening details.
4.2	Establish and Maintain a Secure Configuration Process for Network Infrastructure	Establish and maintain a documented secure configuration process for network devices. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	Apply secure configuration practices to network infrastructure that carries MCP traffic, including reverse proxies, load balancers, and Domain Name System (DNS) infrastructure. Ensure TLS termination, header forwarding, and session affinity configurations support Streamable HTTP requirements. Ensure proxy and gateway configurations preserve required auth headers and do not introduce insecure header rewriting or caching of sensitive responses.
4.3	Configure Automatic Session Locking on Enterprise Assets	Configure automatic session locking on enterprise assets after a defined period of inactivity. For general purpose operating systems, the period must not exceed 15 minutes. For mobile end-user devices, the period must not exceed 2 minutes.	●	●	●	No Additional MCP Guidance
4.4	Implement and Manage a Firewall on Servers	Implement and manage a firewall on servers, where supported. Example implementations include a virtual firewall, operating system firewall, or a third-party firewall agent.	●	●	●	Restrict inbound access to approved clients and/or the enterprise gateway, and enforce outbound egress allowlists to only required back-end systems, identity providers, and logging destinations.
4.5	Implement and Manage a Firewall on End-User Devices	Implement and manage a host-based firewall or port-filtering tool on end-user devices, with a default-deny rule that drops all traffic except those services and ports that are explicitly allowed.	●	●	●	Apply host firewall controls to endpoints running MCP hosts and clients to restrict unexpected inbound listeners and limit outbound access to approved MCP servers and back-end services.
4.6	Securely Manage Enterprise Assets and Software	Securely manage enterprise assets and software. Example implementations include managing configuration through version-controlled Infrastructure-as-Code (IaC) and accessing administrative interfaces over secure network protocols, such as Secure Shell (SSH) and Hypertext Transfer Protocol Secure (HTTPS). Do not use insecure management protocols, such as Telnet (Teletype Network) and HTTP, unless operationally essential.	●	●	●	Deploy MCP servers through controlled, repeatable processes that include integrity verification and approval before activation. Use an enterprise MCP registry to manage approved and vetted servers, and prohibit ad hoc installation of community or unvetted servers in production environments. Apply change control to server deployments, including updates that affect capabilities, OAuth settings, or data source connections. Treat changes to declared tools, resources, and prompts as configuration changes requiring review, approval, and re-validation before promotion to production.

CIS Control 4: Secure Configuration of Enterprise Assets and Software

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
4.7	Manage Default Accounts on Enterprise Assets and Software	Manage default accounts on enterprise assets and software, such as root, administrator, and other pre-configured vendor accounts. Example implementations can include: disabling default accounts or making them unusable.	●	●	●	No Additional MCP Guidance
4.8	Uninstall or Disable Unnecessary Services on Enterprise Assets and Software	Uninstall or disable unnecessary services on enterprise assets and software, such as an unused file sharing service, web application module, or service function.		●	●	Expose only the tools, resources, and prompts required as defined in the approved enterprise MCP registry entry. Remove overly broad tools and limit resource URI patterns to the registry-validated baseline. Do not expose unnecessary debugging tools or overly permissive resource patterns; implement required security logging centrally while minimizing sensitive content in logs. Reducing declared capabilities by enforcing that the tools, resources, and prompts a server exposes during initialization match its registered profile lowers attack surface.
4.9	Configure Trusted DNS Servers on Enterprise Assets	Configure trusted DNS servers on network infrastructure. Example implementations include configuring network devices to use enterprise-controlled DNS servers and/or reputable externally accessible DNS servers.		●	●	Configure all assets running MCP hosts or clients to use enterprise-managed or trusted DNS resolvers. Untrusted DNS can redirect MCP clients to malicious servers or substitute illegitimate authorization server discovery endpoints, undermining the OAuth authorization chain before a connection is established. DNS integrity is a prerequisite for secure server discovery, token issuance, and capability negotiation in Streamable HTTP deployments. For deployments that use registries or dynamic server discovery, treat DNS integrity as part of the discovery trust chain and monitor for unexpected resolution changes.
4.10	Enforce Automatic Device Lockout on Portable End-User Devices	Enforce automatic device lockout following a predetermined threshold of local failed authentication attempts on portable end-user devices, where supported. For laptops, do not allow more than 20 failed authentication attempts; for tablets and smartphones, no more than 10 failed authentication attempts. Example implementations include Microsoft® InTune Device Lock and Apple® Configuration Profile maxFailedAttempts.		●	●	No Additional MCP Guidance
4.11	Enforce Remote Wipe Capability on Portable End-User Devices	Remotely wipe enterprise data from enterprise-owned portable end-user devices when deemed appropriate such as lost or stolen devices, or when an individual no longer supports the enterprise.		●	●	No Additional MCP Guidance
4.12	Separate Enterprise Workspaces on Mobile End-User Devices	Ensure separate enterprise workspaces are used on mobile end-user devices, where supported. Example implementations include using an Apple® Configuration Profile or Android™ Work Profile to separate enterprise applications and data from personal applications and data.			●	No Additional MCP Guidance

Additional MCP Considerations

- Include a tested rollback plan for configuration changes using the enterprise MCP Registry as the versioned repository for the “known-good” configuration state, with special attention to changes that affect authorization, scope handling, or gateway policy.
- Validate configuration drift on a fixed cadence and after each change. Confirm deployed servers and gateways match the approved registry baseline for transport, authorization, and logging redaction controls.
- Prefer centralized gateways for production deployments to align with approved transport, authorization, and policy baselines enforced via the enterprise MCP Registry.
- Where Origin validation is applicable (e.g., browser-mediated clients), reject requests with missing or unexpected Origin values and enforce an allowlist; for non-browser clients, use mutual TLS, signed workload identity, or equivalent controls at the gateway. Enforce protocol and version negotiation per the MCP specification and reject requests that do not meet the agreed version requirements.
- When the Tasks capability is enabled, configure Time-to-Live (TTL) policies for task state to prevent unbounded resource accumulation. Ensure task state is cleaned up on session termination, and that completed or cancelled tasks are purged within defined retention windows.
- Align task state retention and purge windows to the enterprise data retention policy defined under Control 3.
- Use the tool and resource schemas stored in the MCP Registry as an active configuration filter at the gateway or host level. Block any server attempts to execute tools or provide resources that deviate from the registered configuration schema.
- Utilize the registry to maintain an “allowlist” of authorized environment variables for each server. Prevent the injection of unauthorized credentials or configuration overrides during server startup that are not documented in the registry.

Control 5: Account Management

Use processes and tools to assign and manage authorization to credentials for user accounts, including administrator accounts, as well as service accounts, to enterprise assets and software.

MCP Applicability

Identity management is critical in MCP, which distributes trust across hosts, servers, gateways, and downstream tools. These layers rely on human and non-human identities and often use long-lived authorizations that, if not governed, create durable access paths to sensitive systems. Account management must ensure identity binding so that tool execution and resource access can be attributed to a human user or approved workload identity end-to-end. Manage OAuth grants separately from downstream tool credentials; compromise of either can create durable access paths.

Key account management considerations in MCP environments include:

- MCP relies on human and non-human identities across the stack, including OAuth client registrations, service accounts and workload identities used by hosts, servers and gateways. Tool integrations also introduce downstream credentials such as API keys and service tokens, and these must be stored in approved secrets systems and never embedded in code.
- The authorization grant life cycle requires close control because OAuth authorizations can persist through refresh tokens and consented scopes. Therefore, it is important to track which identities and clients have active grants to MCP servers, and ensure that offboarding and rotation revoke access quickly.
- Privilege can concentrate through tools because service accounts and downstream credentials used by tools often carry broad permissions. Applying least privilege, separating high-impact tool permissions, and reviewing access for drift on a fixed cadence can all help follow the principle of least privilege.

Safeguards

CIS Control 5: Account Management

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
5.1	Establish and Maintain an Inventory of Accounts	Establish and maintain an inventory of all accounts managed in the enterprise. The inventory must at a minimum include user, administrator, and service accounts. The inventory, at a minimum, should contain the person's name, username, start/stop dates, and department. Validate that all active accounts are authorized, on a recurring schedule at a minimum quarterly, or more frequently.	●	●	●	Maintain an inventory of all user and service accounts that install, operate, or run MCP servers. For Streamable HTTP, include service accounts tied to authorization servers and resource owner accounts with OAuth grants. For stdio, include endpoint accounts that run servers. Link each identity to the MCP component it serves (host, client, server, gateway), the tools and resources it can access, and its risk tier (read-only vs. write vs. irreversible).

CIS Control 5: Account Management

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
5.2	Use Unique Passwords	Use unique passwords for all enterprise assets. Best practice implementation includes, at a minimum, an 8-character password for accounts using Multi-Factor Authentication (MFA) and a 14-character password for accounts not using MFA.	●	●	●	Use strong, unique credentials for all MCP administrative and service identities, preferring federated identity, workload identity, and short-lived credentials over static secrets where possible. Where passwordless methods cannot be implemented, all credentials (including passwords, keys, and tokens) must be rotated on a defined schedule to maintain account security. For Streamable HTTP servers, this includes authorization server accounts, gateway administration accounts, and OAuth client credentials for confidential clients. Client ID Metadata Documents may be used as an alternative to pre-registration, where supported by the chosen authorization server. Service accounts accessing backend APIs through tools must use strong authentication mechanisms.
5.3	Disable Dormant Accounts	Delete or disable any dormant accounts after a period of 45 days of inactivity, where supported.	●	●	●	Disable user and service accounts that no longer require MCP access, and revoke OAuth access and refresh tokens for any account marked dormant. Use an inactivity threshold without MCP authentication or tool invocation to identify dormant accounts for review and disabling. Identify dormant access by both authentication events and tool invocation history; revoke grants even if the user account remains active. Include role change as a revocation review trigger alongside offboarding and dormancy: a change in job function can invalidate existing OAuth grants and tool access paths without the account becoming inactive.
5.4	Restrict Administrator Privileges to Dedicated Administrator Accounts	Restrict administrator privileges to dedicated administrator accounts on enterprise assets. Conduct general computing activities, such as internet browsing, email, and productivity suite use, from the user's primary, non-privileged account.	●	●	●	Use dedicated accounts for all administrative activities involving MCP infrastructure, including gateway configuration, authorization server administration, server registry management, and OAuth client registration. Administrative and operational privileges must be handled on entirely separate accounts. Protect configuration files that define server capabilities and authorization metadata to prevent unauthorized modification. Require MFA and privileged access management (PAM) controls for administrative accounts, including break-glass procedures with enhanced logging.

CIS Control 5: Account Management

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
5.5	Establish and Maintain an Inventory of Service Accounts	Establish and maintain an inventory of service accounts. The inventory, at a minimum, must contain department owner, review date, and purpose. Perform service account reviews to validate that all active accounts are authorized, on a recurring schedule at a minimum quarterly, or more frequently.		●	●	Inventory all service accounts supporting MCP deployments: accounts used by server processes to access backend APIs, gateways connecting to authorization servers, automated deployment processes, and server-to-server operations such as sampling requests. For each account, record the business purpose, account owner or responsible department, associated components, systems accessed, credential rotation schedule, authorized scopes, environment (e.g., development, staging, production), the tools and resources the account is authorized to access, and the approved secrets system or location used to store credentials. Cross-environment credential reuse must be explicitly prohibited. Conduct reviews at least a quarterly cadence to verify that each account remains necessary, correctly scoped, and aligned with current operational requirements. Reduce privileges for high-impact tool accounts between reviews when scope drift is detected rather than waiting for the next scheduled cycle.
5.6	Centralize Account Management	Centralize account management through a directory or identity service.		●	●	Integrate Streamable HTTP deployments with enterprise authorization servers. Configure OAuth flows to authenticate through enterprise identity providers. Authorization servers should use enterprise IdP and standard discovery mechanisms where available. stdio deployments inherit the account management model of their host endpoints. Third-party servers accessed through gateways can use centralized authorization with Protected Resource Metadata discovery. Prefer centralized authorization via gateway or shared auth services to avoid per-server credential sprawl and inconsistent policy.

Additional MCP Considerations

- Separate identities used for tool execution from those used for administration and avoid reusing broad service accounts across multiple tools or servers.
- Retrieve credentials at runtime from an approved enterprise secrets manager with rotation and access auditing. For stdio deployments, pass a minimal environment to subprocesses so that sensitive variables are not exposed.
- Prohibit shared downstream service accounts across multiple tools or servers unless explicitly approved; shared identities defeat attribution and blast-radius control.

Control 6: Access Control Management

Use processes and tools to create, assign, manage, and revoke access credentials and privileges for user, administrator, and service accounts for enterprise assets and software.

MCP Applicability

Access control in MCP environments functions as a sequence of checkpoints rather than a single gateway. Each checkpoint governs a distinct aspect of the interaction: which servers an AI agent may contact, which tools it may invoke, and the specific operations those tools are authorized to perform once executed. Because MCP relies on multiple interconnected components, the objective is not only to verify who can authenticate but also to define how far an authenticated identity can act within the ecosystem. The principal security challenges emerge at these internal boundaries.

Key access control considerations in MCP environments include:

- Governing server access, tool invocation, and resource access separately. OAuth scopes support coarse-grained restrictions, but high-impact tools often require additional checks in tool handlers or gateways to enforce business rules. For agent-driven workflows, implement tool-level access controls that restrict which tools may be invoked based on policy-defined workflow scope and the authenticated identity; do not rely on model- or agent-declared intent for authorization decisions, since identity providers typically cannot enforce this level of granularity. Model output and client requests are never authorization decisions; servers and gateways must enforce policy deterministically.
- A host may connect to multiple servers with different ownership and risk profiles, making it important to ensure that access to one server does not grant implicit access to others. Isolating third-party servers through allowlists and policy controls helps prevent unintended cross-access and reduces exposure to higher-risk systems.
- Tools act as delegated access paths into backend systems, so their execution identities should use least privilege. Stronger controls are especially necessary for tools that modify data or trigger administrative actions.
- In Streamable HTTP, clients fetch Protected Resource Metadata from `/.well-known/oauth-protected-resource` to learn the authorization server, then discover that server via its own well-known endpoint. Misconfigured metadata can redirect authentication to an attacker. Validate these endpoints and protect them from unauthorized changes.`
- Elicitation must be access controlled: specify which servers may issue requests via the client, what input they may solicit, and how responses are handled. Because it creates a direct server-to-user channel, restrict Elicitation to approved servers and reviews prompt patterns to reduce credential harvesting, social engineering, and consent manipulation.
- When servers declare the Tasks capability, access control must extend to task life cycle operations. Clients should only be able to retrieve results, check status, or cancel tasks they created. Servers must enforce per-client task isolation and validate authorization on ``tasks/list``, ``tasks/result``, and ``tasks/cancel`` operations.

Safeguards

CIS Control 6: Access Control Management

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
6.1	Establish an Access Granting Process	Establish and follow a documented process, preferably automated, for granting access to enterprise assets upon new hire or role change of a user.	●	●	●	Implement a formal, documented process for granting access to MCP servers, tools, and data sources. Authenticate the client on every request and enforce authorization that matches the formally approved grant. Deny or revoke any request outside the approved tool, resource, scope, or time window. For Streamable HTTP servers, require OAuth access tokens with PKCE; clients must include the `resource` parameter in token requests to bind each token to a specific MCP server, and servers must reject tokens whose audience does not match their resource identifier. Validate issuer, expiry, and scopes on every request. For stdio servers, validate the server executable against an approved allowlist using integrity checks such as cryptographic hashes or path-based restrictions. For proxy servers connecting to third-party APIs, implement per-client consent verification before initiating authorization flows. To counter tool poisoning, the consent process must include 'full tool transparency,' displaying the complete, un-summarized tool manifest (including descriptions and parameters) to the user before a server is authorized.
6.2	Establish an Access Revoking Process	Establish and follow a process, preferably automated, for revoking access to enterprise assets, through disabling accounts immediately upon termination, rights revocation, or role change of a user. Disabling accounts, instead of deleting accounts, may be necessary to preserve audit trails.	●	●	●	Require explicit deprovisioning of MCP access during offboarding by revoking authorizations and disabling refresh mechanisms, terminating active sessions, removing Client ID Metadata Documents where used, and for stdio deployments terminating server processes and removing server executables. Cancel outstanding tasks associated with the revoked identity where the server supports task cancellation, using `tasks/cancel` with the relevant task ID. Task handles issued prior to revocation may otherwise allow result retrieval to continue after the authorization grant has been terminated. Invalidate cached authorization decisions and terminate gateway sessions associated with revoked identities.
6.3	Require MFA for Externally-Exposed Applications	Require all externally-exposed enterprise or third-party applications to enforce MFA, where supported. Enforcing MFA through a directory service or single sign-on (SSO) provider is a satisfactory implementation of this Safeguard.	●	●	●	Require MFA for human users through the enterprise IdP on all internet-accessible MCP servers. For non-interactive and gateway-to-server access, require mutual TLS between client and server and bind OAuth tokens to the client certificate to prevent replay; where mTLS is not feasible, require sender-constrained tokens such as DPoP (Demonstration of Proof-of-Possession). Authorization servers must enforce PKCE (Proof Key for Code Exchange) for authorization code flows. For automated and agent-driven workflows that cannot use MFA, require OAuth client credentials or Personal Access Tokens stored in an enterprise secrets manager with audited access and defined rotation policies. Enforce TLS 1.2 or later with Origin validation for Streamable HTTP. stdio deployments inherit endpoint MFA requirements.

CIS Control 6: Access Control Management

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
6.4	Require MFA for Remote Network Access	Require MFA for remote network access.	●	●	●	No Additional MCP Guidance
6.5	Require MFA for Administrative Access	Require MFA for all administrative access accounts, where supported, on all enterprise assets, whether managed on-site or through a service provider.	●	●	●	Require MFA for administrative access to MCP infrastructure, including gateway administration, authorization server configuration, server registry management, and OAuth client registration.
6.6	Establish and Maintain an Inventory of Authentication and Authorization Systems	Establish and maintain an inventory of the enterprise's authentication and authorization systems, including those hosted on-site or at a remote service provider. Review and update the inventory, at a minimum, annually, or more frequently.		●	●	Inventory authentication and authorization systems that support MCP, including authorization servers, gateways, identity providers, and discovery endpoints (e.g., Protected Resource Metadata, Authorization Server Metadata, and OpenID Connect Discovery). Record OAuth endpoints, supported flows, audience binding approach, and integrations. Maintain an inventory of OAuth client registrations, including owner, approved scopes, and credential rotation requirements.
6.7	Centralize Access Control	Centralize access control for all enterprise assets through a directory service or SSO provider, where supported.		●	●	Use gateway-based authorization to centralize policy enforcement, logging, and OAuth flow management. Consolidate authorization through gateways rather than implementing separate OAuth logic per server. Gateways can enforce consistent Origin validation, rate limiting, and session management. Gateways should support deny-by-default policy and a rapid kill switch to disable high-risk tools or servers during investigation.
6.8	Define and Maintain Role-Based Access Control	Define and maintain role-based access control, through determining and documenting the access rights necessary for each role within the enterprise to successfully carry out its assigned duties. Perform access control reviews of enterprise assets to validate that all privileges are authorized, on a recurring schedule at a minimum annually, or more frequently.			●	Conduct routine reviews of MCP roles, OAuth scopes, and actual tool usage. Revoke unused or excessive permissions and adjust assignments to least privilege. Define and maintain roles that govern access to MCP tools and resources, and map OAuth scopes to those roles. Clients must request only the minimum scopes required for their intended operations, using server-advertised scope information where available. Update role definitions and scope mappings as tools and resources change. RBAC policies should grant permissions at the level of individual tools or resources and specific action types, including read, write, and irreversible operations, rather than through broad scope assignments.

Additional MCP Considerations

- Define a standard method for assigning tool permissions, such as grouping tools by risk tier and requiring additional controls for high-risk tools. To prevent scenarios where the output of one tool inadvertently triggers or manipulates the execution of a subsequent tool, a vulnerability known as tool interference, explicitly isolate the execution context between distinct tool calls. For agentic systems where one tool's output serves as another's input, enforce differentiated Human-in-the-Loop (HITL) requirements that scale the level of human oversight (e.g., mandatory approval versus simple notification) based on the cumulative risk of the tool sequence.
- Require formal approval before granting additional scopes; record the approval with Approver's identity, reason, timestamp, and scope changes. Validate that 'listChanged' notifications do not introduce 'shadow tools' that bypass the initial scope approval.
- Where feasible, centralize authorization policy enforcement through a gateway or enterprise identity integration to prevent per-server configuration drift, reduce repeated consent prompts, and maintain consistent authorization outcomes.
- Confirm that downstream systems enforce their own authorization checks; do not rely on MCP scopes alone to satisfy business rules.
- Implement periodic capability validation to ensure servers have not expanded declared tools or resources outside approved baselines.
- Do not rely on tool annotations (e.g., `readOnlyHint/destructiveHint`) or model prompts to enforce access control; treat annotations as untrusted metadata.
- For Streamable HTTP, bind session IDs to user-specific information (e.g., a hashed user ID) at initialization. Verify this binding on every subsequent request to prevent session hijacking or cross-session impersonation.

Control 7: Continuous Vulnerability Management

Develop a plan to continuously assess and track vulnerabilities on all enterprise assets within the enterprise's infrastructure, in order to remediate, and minimize, the window of opportunity for attackers. Monitor public and private industry sources for new threat and vulnerability information.

MCP Applicability

Vulnerability management in MCP environments goes beyond routine patching because the protocol relies on custom servers, shared libraries, SDKs, and third-party components that evolve at different rates. MCP deployments often depend on rapidly shifting ecosystems, where new features, tooling, and integrations can introduce gaps long before traditional scanning tools catch them. MCP servers, gateways, and tool wrappers are software assets requiring continuous vulnerability management.

Key vulnerability management considerations in MCP environments include:

- MCP servers and tool wrappers from registries and package ecosystems introduce third-party dependencies that expand patching scope and tracking requirements.
- Deployments rely on SDKs across multiple languages plus shared dependencies for JSON-RPC, transport, and authorization, increasing the risk of inconsistent patch levels across environments.
- Specification and SDK updates can change transport and authorization behavior. Treat protocol upgrades as security-relevant changes requiring validation of configuration and authorization behavior.
- Treat MCP spec and SDK upgrades and capability changes as security-relevant events requiring regression testing of authorization, Origin and transport controls, and tool boundaries.

Safeguards

CIS Control 7: Continuous Vulnerability Management

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
7.1	Establish and Maintain a Vulnerability Management Process	Establish and maintain a documented vulnerability management process for enterprise assets. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	Implement vulnerability management for MCP servers, client libraries, OAuth implementations, JSON-RPC frameworks, and transport layers. Be aware of security advisories for MCP server runtimes, client SDKs, gateway plugins, specification updates, third-party server disclosures, and dependency vulnerabilities, and apply updates or compensating controls as issues are disclosed.
7.2	Establish and Maintain a Remediation Process	Establish and maintain a risk-based remediation strategy documented in a remediation process, with monthly, or more frequent, reviews.	●	●	●	Define SLAs for remediating MCP vulnerabilities by severity. Prioritize critical vulnerabilities in internet-accessible Streamable HTTP servers, such as Origin validation bypass, OAuth token theft, or session hijacking. Remediation actions may include patching components, reconfiguring authorization metadata, or temporarily disabling vulnerable tools until fixes are available.

CIS Control 7: Continuous Vulnerability Management

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
7.3	Perform Automated Operating System Patch Management	Perform operating system updates on enterprise assets through automated patch management on a monthly, or more frequent, basis.	●	●	●	No Additional MCP Guidance
7.4	Perform Automated Application Patch Management	Perform application updates on enterprise assets through automated patch management on a monthly, or more frequent, basis.	●	●	●	Automate patching for MCP components including server frameworks, OAuth libraries, JSON-RPC implementations, and transport layers. Use package managers to track and update dependencies. Community servers may not support automated updates, requiring manual monitoring of releases and advisories. Prioritize updates addressing token handling, authorization metadata, session management, tool execution path security, and MCP specification compliance issues, as vulnerabilities in these areas have the highest potential for authorization bypass and privilege escalation.
7.5	Perform Automated Vulnerability Scans of Internal Enterprise Assets	Perform automated vulnerability scans of internal enterprise assets on a quarterly, or more frequent, basis. Conduct both authenticated and unauthenticated scans.		●	●	Scan endpoints running stdio servers, hosts running Streamable HTTP servers, gateways, and authorization servers. Identify outdated versions, vulnerable dependencies, and misconfigurations. Standard vulnerability scanners do not cover MCP-specific settings. Add custom checks for bind address exposure, TLS configuration, Origin validation (where applicable), PKCE enforcement, token audience validation, capability drift, and session handling.
7.6	Perform Automated Vulnerability Scans of Externally-Exposed Enterprise Assets	Perform automated vulnerability scans of externally-exposed enterprise assets. Perform scans on a monthly, or more frequent, basis.		●	●	Scan externally accessible MCP servers and gateways for OAuth misconfigurations such as missing PKCE, weak scope enforcement, and token passthrough, as well as TLS, Origin validation, and authorization policy issues. Include testing for unauthorized tool invocation, scope escalation, and correct Protected Resource Metadata. While some commercial scanners are beginning to include MCP protocol compliance checks, supplement with custom checks for MCP-specific exposures until tooling coverage matures. Scan third-party MCP server artifacts and container images during intake, including SBOM review and malware scanning, before approval for production use.
7.7	Remediate Detected Vulnerabilities	Remediate detected vulnerabilities in software through processes and tooling on a monthly, or more frequent, basis, based on the remediation process.		●	●	Remediate vulnerabilities based on risk. Critical issues involving tool authorization, OAuth token handling, Origin validation, or JSON-RPC compliance require immediate action. Track progress and verify fixes through rescanning. When patches are unavailable, record compensating controls such as disabling affected tools or restricting network access.

Additional MCP Considerations

- Monitor advisories for MCP SDKs, common tooling, and package ecosystems, not only traditional OS and infrastructure sources.
- Treat protocol and SDK upgrades as security-relevant changes requiring testing of authorization and transport behavior.
- For third-party servers, define minimum maintenance expectations including patch timelines and a process to suspend or remove unmaintained servers.
- If maintenance expectations are not met, the default action is to disable or remove the server from production allowlists.

Control 8: Audit Log Management

Collect, alert, review, and retain audit logs of events that could help detect, understand, or recover from an attack.

MCP Applicability

Unlike traditional applications, MCP deployments emit protocol-level events through the protocol's logging utility in addition to standard infrastructure logs, creating a richer stream of information to monitor. These structured notifications follow syslog severity levels and can include optional logger names and JSON metadata, which produces audit signals useful for spotting misuse of tool execution, sensitive data access, unexpected tool activity, and unusual access patterns that may be missed by infrastructure logs alone.

Key audit logging considerations in MCP environments include:

- Recording tool discovery, invocations, resource retrievals, and capability changes with sufficient context is important to identify the server, tool or Uniform Resource Identifier (URI) involved, the outcome, and the identity used.
- Capturing session life cycle events and using JSON-RPC request IDs to correlate requests and responses across components supports investigation and replay analysis.
- Operational logs can expose credentials or personally identifiable information (PII) if tool parameters are logged verbatim. Design MCP logging so that credentials, secrets, and PII are excluded by default through redaction and field-level controls, while preserving investigative value through identifiers and correlation metadata.
- Define a minimum MCP audit schema so that investigations can correlate identity → server → tool/resource → action → outcome across components.
- Task life cycle events, including task creation, status transitions (working, completed, failed, cancelled), result retrieval, and cancellation, must be logged with sufficient context to tie each event to the originating session, identity, and tool invocation. Gaps in task life cycle logging create blind spots where long-running operations complete or fail without a correlated audit record, hindering investigation of delayed or asynchronous tool behavior.

Safeguards

CIS Control 8: Audit Log Management

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
8.1	Establish and Maintain an Audit Log Management Process	Establish and maintain a documented audit log management process that defines the enterprise's logging requirements. At a minimum, address the collection, review, and retention of audit logs for enterprise assets. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	Update the enterprise audit log management process to include MCP-specific log sources and requirements, including protocol events (tool and resource access, capability changes, OAuth and session life cycle events), correlation identifiers, and redaction rules.
8.2	Collect Audit Logs	Collect audit logs. Ensure that logging, per the enterprise's audit log management process, has been enabled across enterprise assets.	●	●	●	Collect audit logs for MCP initialization, capability negotiation, tool invocation, resource retrieval, prompt expansion, OAuth token events, session life cycle, and JSON-RPC errors. Include user identity where available and MCP client identity. Capture the strongest available identity signal per session and update logging as client identity mechanisms mature. Servers that declare the logging capability must emit structured notifications with severity and logger names. Log capability baselines (tools, resources, prompts) at initialization and any subsequent capability changes as high-signal audit events.

CIS Control 8: Audit Log Management

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
8.3	Ensure Adequate Audit Log Storage	Ensure that logging destinations maintain adequate storage to comply with the enterprise's audit log management process.	●	●	●	Size and monitor audit log storage to support the enterprise audit log management process for MCP components (clients, servers, gateways, and supporting auth infrastructure). Account for MCP protocol event volume, including initialization and capability negotiation, tool and resource operations, OAuth and session life cycle events, and JSON-RPC errors.
8.4	Standardize Time Synchronization	Standardize time synchronization. Configure at least two synchronized time sources across enterprise assets, where supported.		●	●	Standardize time synchronization across MCP clients, servers, gateways, and supporting authorization infrastructure to support consistent timestamps for audit log correlation and investigations.
8.5	Collect Detailed Audit Logs	Configure detailed audit logging for enterprise assets containing sensitive data. Include event source, date, username, timestamp, source addresses, destination addresses, and other useful elements that could assist in a forensic investigation.		●	●	Use consistent formats, timestamps, and correlation identifiers across components. Include operation type, identity, tool or resource identifier, result status, message ID, and error details. Where supported, configure verbosity via controlled administrative policy, and restrict who can change log levels, to prevent excessive sensitive data capture.
8.6	Collect DNS Query Audit Logs	Collect DNS query audit logs on enterprise assets, where appropriate and supported.		●	●	No Additional MCP Guidance
8.7	Collect URL Request Audit Logs	Collect URL request audit logs on enterprise assets, where appropriate and supported.		●	●	Collect URL request logs for MCP tool egress and server and gateway outbound calls to providers and registries.
8.8	Collect Command-Line Audit Logs	Collect command-line audit logs. Example implementations include collecting audit logs from PowerShell®, BASH™, and remote administrative terminals.		●	●	Collect command-line logs on endpoints hosting stdio servers and on servers executing tools that invoke shells and scripts.
8.9	Centralize Audit Logs	Centralize, to the extent possible, audit log collection and retention across enterprise assets in accordance with the documented audit log management process. Example implementations primarily include leveraging a SIEM tool to centralize multiple log sources.		●	●	Centralize logs from gateways, Streamable HTTP servers, and stdio servers where clients capture `stderr.` Integrate MCP logs with enterprise SIEM before components enter production to enable event correlation, pattern detection, and security monitoring. Correlate logs with declared capabilities to identify deviations from approved baselines.
8.10	Retain Audit Logs	Retain audit logs across enterprise assets for a minimum of 90 days.		●	●	Define MCP-specific retention for audit logs, registry snapshots, authorization policy versions, and server configuration history. Retain tool and resource access, capability changes, OAuth events, sampling requests, and session life cycle data for periods aligned to incident response, audit, and data sensitivity.
8.11	Conduct Audit Log Reviews	Conduct reviews of audit logs to detect anomalies or abnormal events that could indicate a potential threat. Conduct reviews on a weekly, or more frequent, basis.		●	●	Conduct regular audit log reviews for MCP interactions to detect anomalies, unauthorized activity, and deviations from approved capabilities. Verify that logs contain validated data fields, appropriate severity levels, correlation identifiers, and structured JSON-RPC records. Confirm that logs exclude credentials, secrets, and PII.

CIS Control 8: Audit Log Management

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
8.12	Collect Service Provider Logs	Collect service provider logs, where supported. Example implementations include collecting authentication and authorization events, data creation and disposal events, and user management events.				Collect logs from third-party MCP servers, authorization servers, and cloud-hosted gateways. Logs support incident investigations and security monitoring. Verify service agreements include retention commitments and access procedures.

Additional MCP Considerations

- Define an MCP audit event taxonomy including server identity, tool name, resource URI, outcomes, and scopes used.
- Design redaction so that logs preserve investigative value without storing sensitive parameters, and define redaction rules.
- Ensure log correlation works across client, gateway, and server components using session identifiers and request IDs.
- Log Elicitation requests and user responses, including the requesting server identity, input schema, and whether the user accepted or declined the prompt. Elicitation creates a direct user interaction channel that should be auditable.
- Record human approval and denial decisions for tool invocations with identity, reason, timestamp, and tool parameters to support audit and incident investigation.
- When Tasks are enabled, log task creation, status transitions, result retrieval, and cancellation events. Include task IDs in correlation data to support tracing of long-running operations across components.
- Apply two-tier logging as defined in Control 3 Safeguard 3.14.

Control 9: Email and Web Browser Protections

Improve protections and detections of threats from email and web vectors, as these are opportunities for attackers to manipulate human behavior through direct engagement.

MCP Applicability

MCP is not an email or browser protocol, but MCP-connected tools and resources can ingest untrusted web and messaging content and place it into an LLM context window. Content from web, email, and repositories can be used for indirect prompt injection, tool misuse, or data exfiltration through downstream actions. Treat web, email, and repository content as untrusted inputs and apply screening, provenance tagging, and constrained summarization before injecting into context.

Key MCP-specific implications include:

- Untrusted content in context arises when tools and resources that fetch web pages, issues, documents, or messages import malicious instructions that influence tool selection and execution.
- Using controlled egress through an MCP gateway or proxy reduces exposure by allowing DNS and URL filtering at egress, and gateway policies that tag or segregate MCP traffic from general browsing further support this separation.
- Treating browser, email, and retrieval tools as higher risk helps ensure these tools receive tighter approval, monitoring, and least privilege controls.

Safeguards

CIS Control 9: Email and Web Browser Protections

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
9.1	Ensure Use of Only Fully Supported Browsers and Email Clients	Ensure only fully supported browsers and email clients are allowed to execute in the enterprise, only using the latest version of browsers and email clients provided through the vendor.	●	●	●	MCP servers that wrap web browsing or email processing capabilities must be treated as browser or email clients for the purposes of this Safeguard. Ensure the underlying libraries and runtimes used by these tools are actively maintained, patched, and subject to the same support life cycle requirements as enterprise-approved browsers and email clients. Unsupported or unpatched tool wrappers that fetch web content or process email introduce the same phishing, malicious content, and exploit risks as unpatched end-user clients, with the added risk that compromised content enters model context rather than a human-visible interface.
9.2	Use DNS Filtering Services	Use DNS filtering services on all end-user devices, including remote and on-premises assets, to block access to known malicious domains.	●	●	●	Ensure all DNS queries generated by MCP tools are routed through an approved DNS filtering service. Tools fetching web content or resolving URIs should be subject to DNS-based protection against malicious domains. Where third-party servers are permitted, use DNS policy to restrict access to approved endpoints and alert on resolution attempts for unknown domains.

CIS Control 9: Email and Web Browser Protections

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
9.3	Maintain and Enforce Network-Based URL Filters	Enforce and update network-based URL filters to limit an enterprise asset from connecting to potentially malicious or unapproved websites. Example implementations include category-based filtering, reputation-based filtering, or through the use of block lists. Enforce filters for all enterprise assets.		●	●	Apply network-based URL filtering to MCP tool egress that retrieves web content or calls external HTTP(S) endpoints. Use allowlists for approved categories and destinations, and block high-risk and unapproved destinations to reduce prompt-injection, data exfiltration, and command-and-control paths.
9.4	Restrict Unnecessary or Unauthorized Browser and Email Client Extensions	Restrict, either through uninstalling or disabling, any unauthorized or unnecessary browser or email client plugins, extensions, and add-on applications.		●	●	Restrict unauthorized browser extensions used in MCP workflows (e.g., copilot extensions, admin consoles, developer tooling) and block extensions that can access sensitive content or inject scripts into administrative interfaces.
9.5	Implement DMARC	To lower the chance of spoofed or modified emails from valid domains, implement DMARC policy and verification, starting with implementing the Sender Policy Framework (SPF) and the DomainKeys Identified Mail (DKIM) standards.		●	●	Implement DMARC on enterprise email domains to reduce spoofed emails that target MCP administrators and users. Phishing campaigns may distribute malicious MCP server configuration files, direct recipients to install trojanized server packages, or impersonate internal teams to obtain OAuth consent approvals for unauthorized servers.
9.6	Block Unnecessary File Types	Block unnecessary file types attempting to enter the enterprise's email gateway.		●	●	No Additional MCP Guidance
9.7	Deploy and Maintain Email Server Anti-Malware Protections	Deploy and maintain email server anti-malware protections, such as attachment scanning and/or sandboxing.			●	No Additional MCP Guidance

Additional MCP Considerations

- Route high-risk retrieval through a controlled path such as a gateway, proxy, or constrained egress that enforces registry-defined security policies and destination allowlists, and also ensures that log retrieval metadata including server identity, destination, and outcome are collected for auditing.
- Prefer allowlisted destinations and an approved server catalog for tools that browse, scrape, or read untrusted content.
- Screen content before ingestion for prompt injection and sensitive data, and define rules for what may be passed, summarized, or excluded. Specifically check for malicious text embedded within retrieved web pages, support tickets, or repository files that attempts to supersede the model's system instructions, a vulnerability known as instruction overrides, which is a specific form of prompt injection.
- Enforce web egress for tool-driven HTTP and log tool retrieval separately from user browsing.
- Require approval with least privilege for tools that access mailboxes, repositories, and document stores.
- Do not rely on tool annotations for security decisions (see Control 6).
- Enforce tool risk tiering and approvals using enterprise policy, not server-provided annotations; treat annotations as untrusted metadata.

Control 10: Malware Defenses

Prevent or control the installation, spread, and execution of malicious applications, code, or scripts on enterprise assets.

MCP Applicability

MCP can introduce malware risks in areas traditional systems don't, especially when components run locally or retrieve content that is later executed or processed by privileged runtimes. These risks are heightened when local servers and tool wrappers are installed on endpoints, or when tool runtimes download, execute, or write files. This is particularly relevant for stdio deployments on developer workstations or environments that allow third-party server installation. Treat MCP servers and tool wrappers as software supply-chain risk, where provenance, signing, and intake scanning are as important as endpoint malware controls.

Key malware defense considerations in MCP environments include:

- stdio servers run as local processes under the user context, which means that a malicious package can execute with the same privileges as the host application and access local files and environment variables.
- Servers and tool wrappers from package ecosystems may introduce compromised dependencies into trusted development workflows through supply chain attacks.
- Tools that execute commands, run scripts, or handle downloads can become delivery and execution paths for malware if inputs and execution contexts are not controlled.

Safeguards

CIS Control 10: Malware Defenses

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
10.1	Deploy and Maintain Anti-Malware Software	Deploy and maintain anti-malware software on all enterprise assets.	●	●	●	Deploy anti-malware on endpoints running MCP stdio servers and scan server executables and dependencies before deployment. Malicious servers from registries may execute arbitrary code with user privileges, access environment variables, or read sensitive files. Include MCP installation directories in scheduled scan paths and monitor server processes for suspicious behavior such as unexpected file access, outbound connections, or credential reads. Scan MCP server artifacts and container images during intake and before deployment, not only after installation on endpoints.
10.2	Configure Automatic Anti-Malware Signature Updates	Configure automatic updates for anti-malware signature files on all enterprise assets.	●	●	●	Configure automatic signature and definition updates for anti-malware on all endpoints running MCP stdio servers and systems hosting Streamable HTTP servers. Ensure update schedules are frequent enough to detect newly identified malicious MCP packages from registries. If the enterprise maintains MCP-specific detection rules or EDR prevention policies for server processes (see Safeguards 13.2 and 13.7), review and update them on a defined cadence. Maintain EDR prevention and detection rules specific to MCP server process behaviors (e.g., unexpected network egress, secret access, file enumeration, child process spawning).

CIS Control 10: Malware Defenses

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
10.3	Disable Autorun and Autoplay for Removable Media	Disable autorun and autoplay auto-execute functionality for removable media.				No Additional MCP Guidance
10.4	Configure Automatic Anti-Malware Scanning of Removable Media	Configure anti-malware software to automatically scan removable media.				No Additional MCP Guidance
10.5	Enable Anti-Exploitation Features	Enable anti-exploitation features on enterprise assets and software, where possible, such as Microsoft® Data Execution Prevention (DEP), Windows® Defender Exploit Guard (WDEG), or Apple® System Integrity Protection (SIP) and Gatekeeper™.				Enable OS-level anti-exploitation features on systems running MCP servers, clients, and supporting infrastructure. Apply memory protection, control-flow integrity, and exploit mitigations available on each platform. On Linux, use mandatory access control frameworks, Secure Computing Mode (seccomp) profiles, and capability-dropping to constrain MCP server processes.
10.6	Centrally Manage Anti-Malware Software	Centrally manage anti-malware software.				No Additional MCP Guidance
10.7	Use Behavior-Based Anti-Malware Software	Use behavior-based anti-malware software.				No Additional MCP Guidance

Additional MCP Considerations

- Sandbox or constrain stdio servers and tool runtimes using containers, namespaces, or application sandboxes to limit filesystem, network, and subprocess access. Validate that sandboxed servers cannot access resources outside their declared roots or reach network destinations beyond those required for their function.
- Scan server packages, container images, and dependencies during intake and define a process to quarantine suspicious servers.
- For high-risk environments, require stdio servers to run in a sandboxed or containerized execution boundary by default, with explicit exceptions required.

Control 11: Data Recovery

Establish and maintain data recovery practices sufficient to restore in-scope enterprise assets to a pre-incident and trusted state.

MCP Applicability

MCP recovery planning involves restoring more than just systems; it also requires re-establishing trust in the authorization and capability configuration that supports them. After an incident, bringing a server back from backup may not be enough if tokens, registrations, or tool exposures were affected during the compromise. Recovery must include re-validation of capability baselines, authorization bindings, and logging controls before returning MCP services to production.

Key data recovery considerations in MCP environments include:

- Inability to revoke or invalidate tokens, disable refresh mechanisms, rotate client secrets where applicable, and remove or re-issue authorizations for servers and gateways can allow continued access after a compromise.
- Failure to restore servers, gateways, and supporting configuration from known-good sources, including approved server lists, tool allowlists, authorization and gateway policy versions, and registry snapshots, can result in reintroducing compromised or unauthorized components back into the environment.
- Failure to terminate active sessions, confirm that capability exposure matches approved baselines, and validate that logging and monitoring remain intact can allow unauthorized activity to persist and limit follow-on investigation.

Safeguards

CIS Control 11: Data Recovery

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
11.1	Establish and Maintain a Data Recovery Process	Establish and maintain a documented data recovery process that includes detailed backup procedures. In the process, address the scope of data recovery activities, recovery prioritization, and the security of backup data. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	Extend the enterprise data recovery process to include MCP-specific assets: server and gateway configurations, capability manifests (tools, resources, prompts), authorization policy versions, registry snapshots, and identity and OAuth configurations. Define recovery sequencing that restores identity and policy enforcement layers (IdP, authorization server, and gateway) before restoring tool execution servers, ensuring authorization controls are fully operational before any tool becomes accessible.
11.2	Perform Automated Backups	Perform automated backups of in-scope enterprise assets. Run backups weekly, or more frequently, based on the sensitivity of the data.	●	●	●	Back up MCP server configurations, authorization policies, tool definitions and schemas, resource URI registries, prompt templates, and OAuth client registrations. Include gateway policies and any required session state so that identity, trust, and integration boundaries can be fully restored. Backups must be integrity-verified (hash or signature) and paired with reproducible rebuild steps for servers and gateways.
11.3	Protect Recovery Data	Protect recovery data with equivalent controls to the original data. Reference encryption or data separation, based on requirements.	●	●	●	Protect all MCP recovery data, including backups of MCP server configurations, capability manifests, authorization policies, tool schemas, registries, and identity integrations, using strong access controls and encryption to prevent unauthorized access or tampering.

CIS Control 11: Data Recovery

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
11.4	Establish and Maintain an Isolated Instance of Recovery Data	Establish and maintain an isolated instance of recovery data. Example implementations include, version controlling backup destinations through offline, cloud, or off-site systems or services.	●	●	●	Maintain isolated backup copies of MCP configurations, authorization policies, and registry state in locations not directly accessible from production MCP infrastructure. Store these backups in securely managed, versioned, and integrity-verified locations to ensure the MCP environment can be accurately and safely restored after an incident.
11.5	Test Data Recovery	Test backup recovery quarterly, or more frequently, for a sampling of in-scope enterprise assets.		●	●	Establish recovery time objectives (RTOs) for MCP infrastructure. Critical services require rapid restoration. Test recovery procedures on at least a quarterly cadence for critical MCP infrastructure, including full rebuild from known-good sources and verification that capability negotiation after restore matches the approved baseline. Test backup procedures for servers, gateways, authorization configurations, and session state. Recovery must account for transport differences between stdio and Streamable HTTP. Verify restored servers complete initialization with correct capability negotiation. Include tests for token revocation, refresh-token invalidation, and gateway session termination as part of recovery exercises.

Additional MCP Considerations

- Maintain an incident recovery runbook including revoking authorizations, rotating credentials, disabling servers, and restoring capability baselines.
- Ensure servers and gateways can be rebuilt from known-good sources with reproducible build steps and integrity checks.
- After restore, default to disabling high-risk tools and validate tool exposure, logging, and access control settings before returning services to production, re-enabling high-risk tools only once baselines, authorization, and monitoring are confirmed correct.

Control 12: Network Infrastructure Management

Establish, implement, and actively manage (track, report, correct) network devices, in order to prevent attackers from exploiting vulnerable network services and access points.

MCP Applicability

MCP creates explicit connections between an MCP host and one or more MCP servers. Servers may run locally via stdio or remotely over HTTP transports, which changes how traffic flows through the environment. Network controls limit exposure of MCP services and tightly define which downstream resources tools are permitted to reach. Treat tool egress as controlled network access: default-deny outbound connectivity from tool runtimes except to required destinations.

Key network infrastructure considerations in MCP environments include:

- If HTTP servers and gateways are not segmented, placed behind a reverse proxy, and protected with strong TLS and access controls, then services are exposed to unauthorized access and interception.
- If tool execution can initiate connections to internal or external destinations without segmentation and gateway-mediated egress allowlisting, servers and runtimes may reach services beyond those required for their function.
- If endpoints run localhost HTTP helper services without DNS rebinding protections, Origin validation, and authentication, DNS rebinding risk increases and unintended inbound paths to localhost-bound services may be created. Pure stdio transports are not affected, but any HTTP listener is.

Safeguards

CIS Control 12: Network Infrastructure Management

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
12.1	Ensure Network Infrastructure is Up-to-Date	Ensure network infrastructure is kept up-to-date. Example implementations include running the latest stable release of software and/or using currently supported network as a service (NaaS) offerings. Review software versions monthly, or more frequently, to verify software support.	●	●	●	Maintain network infrastructure supporting MCP Streamable HTTP servers and gateways at current, vendor-supported versions. Ensure reverse proxies, load balancers, and TLS termination points support session management headers and OAuth authorization flows. Migrate legacy SSE servers to Streamable HTTP and replace components that cannot meet current transport requirements.
12.2	Establish and Maintain a Secure Network Architecture	Design and maintain a secure network architecture. A secure network architecture must address segmentation, least privilege, and availability, at a minimum. Example implementations may include documentation, policy, and design components.		●	●	Place servers and gateways in dedicated segments protected by firewall rules. Isolate third-party servers from enterprise-developed servers. Where Origin validation is applicable (e.g., browser-mediated clients), enforce strict Origin allowlists; for non-browser clients, enforce client identity using gateway controls, mTLS, or signed workload identity. Restrict local-only servers to '127.0.0.1'. Record network paths for OAuth authorization flows.

CIS Control 12: Network Infrastructure Management

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
12.3	Securely Manage Network Infrastructure	Securely manage network infrastructure. Example implementations include version-controlled Infrastructure-as-Code (IaC), and the use of secure network protocols, such as SSH and HTTPS.		●	●	Manage network infrastructure supporting MCP through change control, backups, and access restrictions. Record paths between clients, gateways, servers, authorization servers, and backends, including endpoint URLs and OAuth flow requirements.
12.4	Establish and Maintain Architecture Diagram(s)	Establish and maintain architecture diagram(s) and/or other network system documentation. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.		●	●	Maintain diagrams depicting MCP connectivity, data flows, and trust boundaries. Include stdio subprocess communication, Streamable HTTP endpoints, gateways, authorization servers, and backend systems. Describe session management, OAuth discovery mechanisms, and how capability negotiation influences network interactions.
12.5	Centralize Network Authentication, Authorization, and Auditing (AAA)	Centralize network AAA.		●	●	No Additional MCP Guidance
12.6	Use of Secure Network Management and Communication Protocols	Adopt secure network management protocols (e.g., 802.1X) and secure communication protocols (e.g., Wi-Fi Protected Access 2 (WPA2) Enterprise or more secure alternatives).		●	●	No Additional MCP Guidance
12.7	Ensure Remote Devices Utilize a VPN and are Connecting to an Enterprise's AAA Infrastructure	Require users to authenticate to enterprise-managed VPN and authentication services prior to accessing enterprise resources on end-user devices.		●	●	No Additional MCP Guidance
12.8	Establish and Maintain Dedicated Computing Resources for All Administrative Work	Establish and maintain dedicated computing resources, either physically or logically separated, for all administrative tasks or tasks requiring administrative access. The computing resources should be segmented from the enterprise's primary network and not be allowed internet access.			●	Use dedicated workstations for administration of MCP gateways, authorization servers, and server registries. These functions collectively control which tools and capabilities are available to all users and agents in a deployment, making them high-value targets. ● Compromise of an administrative workstation through credential theft, phishing, or a browser exploit on a general-purpose device can cascade into full gateway policy compromise. Administrative workstations used for these functions should not be used for email, browsing, or development work.

Additional MCP Considerations

- Apply controlled egress including allowlists for third-party servers and registries.
- Where endpoints run MCP processes, ensure controls mitigate DNS rebinding and unauthorized localhost access.
- Define how unapproved servers are blocked and define isolation requirements for approved exceptions.
- Restrict registry access and artifact downloads (third-party servers) through controlled egress with allowlists and logging.

Control 13: Network Monitoring and Defense

Operate processes and tooling to establish and maintain comprehensive network monitoring and defense against security threats across the enterprise's network infrastructure and user base.

MCP Applicability

Effective MCP monitoring emphasizes behavioral and capability signals rather than raw network or system data. MCP generates signals that can reveal unusual behavior, especially when tools interact with sensitive systems or trigger high-impact actions. These signals offer valuable insight into whether activity aligns with expected workflows or suggests misuse. Treat capability drift and unusual tool invocation sequences as high-signal indicators of compromise or misuse.

Key monitoring considerations in MCP environments include:

- Failure to detect anomalous tool discovery and invocation, repeated call failures, risky tool sequences, or unexpected capability or tool list changes can allow activity that diverges from expected workflows to go unnoticed.
- Failure to monitor resource retrieval patterns and tool outputs for unexpected volume, unusual endpoints, or sensitive data access outside normal roles or time windows can allow misuse or abnormal activity to go undetected.
- Insufficient monitoring for OAuth events such as new grants, unusual scope requests, and session life cycle indicators can create opportunities for persistence, replay, or misuse across servers and gateways.
- Lack of correlation between network flow data and MCP audit events can make it difficult for investigations to tie outbound connections and downstream system access to specific tool invocations and identities.

Safeguards

CIS Control 13: Network Monitoring and Defense

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
13.1	Centralize Security Event Alerting	Centralize security event alerting across enterprise assets for log correlation and analysis. Best practice implementation requires the use of a SIEM, which includes vendor-defined event correlation alerts. A log analytics platform configured with security-relevant correlation alerts also satisfies this Safeguard.		●	●	Centralize MCP security events from hosts, servers, gateways, and authorization servers. Aggregate tool invocations, capability changes, OAuth events, and session life cycle data to enable correlation across components.
13.2	Deploy a Host-Based Intrusion Detection Solution	Deploy a host-based intrusion detection solution on enterprise assets, where appropriate and/or supported.		●	●	Use host-based detection to identify MCP abuse on endpoints that run stdio servers. Alert on anomalies such as unauthorized server processes, capability changes, unusual file access patterns, and credential reads. While some commercial HIDS (Host-based Intrusion Detection System) products are beginning to include MCP-specific signatures, supplement with custom rules to ensure adequate coverage until the tooling ecosystem matures.

CIS Control 13: Network Monitoring and Defense

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
13.3	Deploy a Network Intrusion Detection Solution	Deploy a network intrusion detection solution on enterprise assets, where appropriate. Example implementations include the use of a Network Intrusion Detection System (NIDS) or equivalent cloud service provider (CSP) service.		●	●	Deploy network-based detection to identify traffic anomalies affecting MCP infrastructure. Not all commercial products include MCP-specific signatures; develop custom rules to monitor for connection rate spikes, traffic to unauthorized destinations, and unencrypted traffic where TLS is required. Content-based detection of MCP protocol semantics requires application-layer inspection (see Safeguard 13.10).
13.4	Perform Traffic Filtering Between Network Segments	Perform traffic filtering between network segments, where appropriate.		●	●	Enforce traffic filtering between MCP network segments: clients to gateways, gateways to MCP servers, and MCP servers to backend systems accessed by tools. Without filtering between these segments, a compromised MCP server has a network path to every backend system its tools connect to, defeating the containment purpose of the gateway architecture. Filtering rules should permit only the specific flows required for each deployment pattern and deny all others by default.
13.5	Manage Access Control for Remote Assets	Manage access control for assets remotely connecting to enterprise resources. Determine amount of access to enterprise resources based on: up-to-date anti-malware software installed, configuration compliance with the enterprise's secure configuration process, and ensuring the operating system and applications are up-to-date.		●	●	No Additional MCP Guidance
13.6	Collect Network Traffic Flow Logs	Collect network traffic flow logs and/or network traffic to review and alert upon from network devices.		●	●	Collect and retain network traffic flow logs for all MCP servers and tool hosts to identify unexpected communication paths, validate egress and segmentation policies, and correlate anomalies detected at the application layer with underlying traffic behaviors.
13.7	Deploy a Host-Based Intrusion Prevention Solution	Deploy a host-based intrusion prevention solution on enterprise assets, where appropriate and/or supported. Example implementations include use of an Endpoint Detection and Response (EDR) client or host-based IPS agent.			●	Deploy endpoint protection capable of blocking unauthorized MCP server execution and suspicious process behavior. Not all commercial EDR products include MCP-specific detections. Configure application control policies to prevent unapproved servers from launching and to block access to credential files outside approved workflows.
13.8	Deploy a Network Intrusion Prevention Solution	Deploy a network intrusion prevention solution, where appropriate. Example implementations include the use of a Network Intrusion Prevention System (NIPS) or equivalent CSP service.			●	Deploy network-based intrusion prevention to actively block detected MCP traffic anomalies. Apply detection rules from Safeguard 13.3 in prevention mode where confidence levels support inline blocking. For Streamable HTTP traffic, gateway-based enforcement provides the most practical inline prevention point. Not all commercial NIPS products include MCP-specific signatures; development of custom prevention rules and reliance on application-layer controls (see Safeguard 13.10) for protocol-level enforcement may be needed.
13.9	Deploy Port-Level Access Control	Deploy port-level access control. Port-level access control utilizes 802.1x, or similar network access control protocols, such as certificates, and may incorporate user and/or device authentication.			●	No Additional MCP Guidance

CIS Control 13: Network Monitoring and Defense

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
13.10	Perform Application Layer Filtering	Perform application layer filtering. Example implementations include a filtering proxy, application layer firewall, or gateway.				Deploy application-aware gateway controls or a Web Application Firewall (WAF) capable of parsing JSON-RPC traffic to enforce protocol compliance, OAuth validation, and tool-invocation policies. Define custom detections for injection attempts, unauthorized tool calls, confused-deputy patterns, and sampling abuse. Network IDS cannot inspect encrypted MCP traffic; rely on gateway telemetry for application-layer security.
13.11	Tune Security Event Alerting Thresholds	Tune security event alerting thresholds monthly, or more frequently.				Tune alerting thresholds based on baseline MCP behavior by environment and role. Adjust sensitivity for tool invocation rates, capability change frequency, and resource access patterns to reduce false positives while detecting anomalies.

Additional MCP Considerations

- Establish detection use cases for sudden invocation increases, repeated failures, unexpected destinations, sensitive resource access, enumeration attempts, bulk export, cross-tenant access, and activity outside normal time windows.
- Baseline normal behavior by environment and role to distinguish expected automation from suspicious activity. Where native tools do not provide sufficient application-layer visibility, enterprises may require gateway telemetry, custom detections, or specialized API and agent security tooling to achieve adequate coverage.
- Correlate audit events with network telemetry to tie connections and data movement to specific servers, tools, identities, and sessions.
- Monitor `notifications/cancelled` and `notifications/progress` messages for anomalies. High cancellation rates from a single server may indicate unstable or manipulated tool behavior. Progress notifications with unusual metadata or frequencies can signal abuse of long-running operations.

Control 14: Security Awareness and Skills Training

Establish and maintain a security awareness program to influence behavior among the workforce to be security conscious and properly skilled to reduce cybersecurity risks to the enterprise.

MCP Applicability

Training for MCP environments goes beyond general awareness because staff make decisions that affect how MCP tools, servers, and authorization paths are introduced and used. Training should prepare people to approve servers, enable tools, grant authorization, recognize manipulation, and follow approval and reporting processes. Completion of role-specific MCP training should be required before granting privileges to approve servers, manage registries, or administer gateways.

Key training considerations in MCP environments include:

- Lack of user and administrator training to verify server sources, follow approval workflows for new servers and tools, and avoid installing or enabling untrusted third-party servers can increase the likelihood of introducing unsafe or unauthorized components.
- When staff are not trained to recognize indirect prompt injection and content-based manipulation from web pages, tickets, documents, or email content, tool use becomes more susceptible to improper influence.
- Gaps in staff training to recognize unusual authorization prompts or scope requests, and to report suspicious tool behavior through established channels, can allow misuse or compromise to go unnoticed.

Safeguards

CIS Control 14: Security Awareness and Skills Training

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
14.1	Establish and Maintain a Security Awareness Program	Establish and maintain a security awareness program. The purpose of a security awareness program is to educate the enterprise's workforce on how to interact with enterprise assets and data in a secure manner. Conduct training at hire and, at a minimum, annually. Review and update content annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	Build role-based curricula aligned to MCP responsibilities such as server approver, tool publisher, registry maintainer, and incident responder. Track behavioral metrics including declined excessive scope requests, time to revoke risky consent, and completion of scenario-based labs. Refresh content when registry governance, capability policies, or specification versions change.
14.2	Train Workforce Members to Recognize Social Engineering Attacks	Train workforce members to recognize social engineering attacks, such as phishing, business email compromise (BEC), pretexting, and tailgating.	●	●	●	Train staff to recognize MCP social engineering vectors including malicious server installation requests, misleading tool descriptions, rug pull attacks, registry typosquatting, and lookalike server names. Include insider risk scenarios such as covert tool creation and data egress via tool outputs.
14.3	Train Workforce Members on Authentication Best Practices	Train workforce members on authentication best practices. Example topics include MFA, password composition, and credential management.	●	●	●	Train administrators on secure OAuth configuration including PKCE enforcement, scope restrictions, and the prohibition on token passthrough. Train end users to verify scope requests before granting consent and to reject unexpected or excessive permission prompts from MCP servers.

CIS Control 14: Security Awareness and Skills Training

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
14.4	Train Workforce on Data Handling Best Practices	Train workforce members on how to identify and properly store, transfer, archive, and destroy sensitive data. This also includes training workforce members on clear screen and desk best practices, such as locking their screen when they step away from their enterprise asset, erasing physical and virtual whiteboards at the end of meetings, and storing data and assets securely.	●	●	●	Train staff on data handling rules for prompts and outputs including classification tags, redaction requirements, and clear guidance against pasting secrets. Include guidance on risky input types such as uploaded files, images, and embedded links that may contain embedded prompt injection or manipulation payloads. Ensure staff understand why environment separation matters, using staging for evaluation and keeping production data strictly within defined boundaries.
14.5	Train Workforce Members on Causes of Unintentional Data Exposure	Train workforce members to be aware of causes for unintentional data exposure. Example topics include mis-delivery of sensitive data, losing a portable end-user device, or publishing data to unintended audiences.	●	●	●	Train workforce on MCP-specific unintentional exposure paths: evaluating and limiting OAuth scope grants at consent time, recognizing when tool outputs containing sensitive data are entering model context or logs, and avoiding copying MCP artifacts such as tool outputs and resource payloads to unauthorized storage or collaboration platforms. Training should be role-differentiated, as administrators approving OAuth grants require different emphasis than developers handling tool outputs or end users interacting with MCP-enabled applications.
14.6	Train Workforce Members on Recognizing and Reporting Security Incidents	Train workforce members to be able to recognize a potential incident and be able to report such an incident.	●	●	●	Train staff to recognize and report MCP-specific incidents including unusual tool behavior, OAuth phishing, prompt injection attempts, and unauthorized server access. Include detection and reporting of shadow MCP use such as unregistered servers and unapproved capability files.
14.7	Train Workforce on How to Identify and Report if Their Enterprise Assets are Missing Security Updates	Train workforce to understand how to verify and report out-of-date software patches or any failures in automated processes and tools. Part of this training should include notifying IT personnel of any failures in automated processes and tools.	●	●	●	Train teams operating MCP servers, gateways, and clients to recognize and report pending security updates to MCP SDKs, JSON-RPC libraries, OAuth libraries, and server runtimes. MCP SDK updates can silently change authorization semantics, fix token validation behavior, or patch capability negotiation logic without obvious external symptoms; personnel who do not know to monitor MCP-specific advisories will run vulnerable components indefinitely.
14.8	Train Workforce on the Dangers of Connecting to and Transmitting Enterprise Data Over Insecure Networks	Train workforce members on the dangers of connecting to, and transmitting data over, insecure networks for enterprise activities. If the enterprise has remote workers, training must include guidance to ensure that all users securely configure their home network infrastructure.	●	●	●	Train remote users and administrators on the risks of operating MCP tooling over insecure networks, including conducting OAuth authorization flows, administering gateways, and developing stdio servers on untrusted connections. Token interception and session hijacking are realistic risks for MCP operations performed over public or uncontrolled networks; Virtual Private Network (VPN) or Zero Trust Network Access (ZTNA) use must be enforced before accessing internal MCP servers, gateways, or authorization infrastructure remotely.
14.9	Conduct Role-Specific Security Awareness and Skills Training	Conduct role-specific security awareness and skills training. Example implementations include secure system administration courses for IT professionals, OWASP® Top 10 vulnerability awareness and prevention training for web application developers, and advanced social engineering awareness training for high-profile roles.		●	●	Train MCP administrators on registry management, capability enablement, rollback procedures, emergency disable actions, and provenance verification including supplier validation, signature verification, and SBOM review. Use sandboxed lab environments for hands-on exercises. Require completion of role-specific training before granting privileges for high-risk actions such as registry writes or third-party server approval.

Additional MCP Considerations

- Ensure training makes it clear that one should never enter credentials or secrets in response to Elicitation prompts.
- Ensure onboarding training covers identifying content-based manipulation such as indirect prompt injection through retrieved documents, web pages, or tickets.
- Train administrators and others who manage MCP configurations that these configurations must never contain secrets of any kind, including passwords, keys, PII, or other sensitive information.
- Train administrators to understand how OAuth grants and tool permissions interact, including how scope approvals propagate across servers and how misconfigured grants can create unintended access paths.
- Train users to recognize suspicious Elicitation prompts. Servers can use Elicitation to request structured input directly from users through the client interface. Train staff to verify that Elicitation requests are expected, come from approved servers, and do not solicit credentials, secrets, or authorization outside established workflows.

Control 15: Service Provider Management

Develop a process to evaluate service providers who hold sensitive data, or are responsible for an enterprise's critical IT platforms or processes, to ensure these providers are protecting those platforms and data appropriately.

MCP Applicability

When MCP environments rely on third-party servers or hosted gateways, those components act as service providers with access to enterprise data and systems. Service provider management should confirm security posture, data handling practices, and the ability to restrict or terminate access. Providers must support rapid disablement (traffic blocking, server removal, token revocation) and evidence preservation during incidents.

Key service provider management considerations in MCP environments include:

- Insufficient visibility into how providers process, store, and retain data from tool parameters, outputs, resource retrievals, or prompts, including their logging practices and redaction controls, can increase the risk of unintended data exposure or retention.
- Unclear processes for how OAuth grants, scopes, and tokens are managed, how access is revoked, and how servers can be disabled or traffic blocked when risk changes can create gaps in controls and delay response to threats.
- Gaps in monitoring and audit log availability, vulnerability management practices, or incident response commitments can hinder coordinated containment and investigation.

Safeguards

CIS Control 15: Service Provider Management

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
15.1	Establish and Maintain an Inventory of Service Providers	Establish and maintain an inventory of service providers. The inventory is to list all known service providers, include classification(s), and designate an enterprise contact for each service provider. Review and update the inventory annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	Inventory third-party servers supplied or hosted by external service providers, tracking their source, registry status, specification version, transport type, and declared capabilities. Ensure the inventory includes security assessment results, approval date, permitted deployment scope, owner, and integration method.
15.2	Establish and Maintain a Service Provider Management Policy	Establish and maintain a service provider management policy. Ensure the policy addresses the classification, inventory, assessment, monitoring, and decommissioning of service providers. Review and update the policy annually, or when significant enterprise changes occur that could impact this Safeguard.		●	●	Define requirements including code signing, vulnerability disclosure processes, secure authorization behavior (PKCE where applicable, audience binding, scope minimization), accurate capability declarations, signed artifacts, vulnerability disclosure processes, and defined patch SLAs. Describe vetting procedures, approval processes, deployment restrictions, monitoring requirements, and incident coordination.
15.3	Classify Service Providers	Classify service providers. Classification consideration may include one or more characteristics, such as data sensitivity, data volume, availability requirements, applicable regulations, inherent risk, and mitigated risk. Update and review classifications annually, or when significant enterprise changes occur that could impact this Safeguard.		●	●	Classify external service providers and third-party MCP servers by the sensitivity of systems and data they access. Apply proportional security requirements based on classification level.

CIS Control 15: Service Provider Management

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
15.4	Ensure Service Provider Contracts Include Security Requirements	Ensure service provider contracts include security requirements. Example requirements may include minimum security program requirements, security incident and/or data breach notification and response, data encryption requirements, and data disposal commitments. These security requirements must be consistent with the enterprise's service provider management policy. Review service provider contracts annually to ensure contracts are not missing security requirements.		●	●	Ensure third-party MCP server providers are required to deliver signed code, record OAuth scopes and capability declarations, follow vulnerability disclosure processes, comply with logging requirements, and support current specification versions. Include contractual terms for data handling and retention, security update timelines, breach notification, log access, rapid authorization revocation during incidents, and exit procedures including workflow migration. Include contract terms for log access and export SLAs, notification of capability changes, and restrictions on retention and secondary use of enterprise data processed through tools and resources.
15.5	Assess Service Providers	Assess service providers consistent with the enterprise's service provider management policy. Assessment scope may vary based on classification(s), and may include review of standardized assessment reports, such as Service Organization Control 2 (SOC 2) and Payment Card Industry (PCI) Attestation of Compliance (AoC), customized questionnaires, or other appropriately rigorous processes. Reassess service providers annually, at a minimum, or with new and renewed contracts.			●	Assess third-party MCP server providers for compliance with security requirements including capability declaration accuracy, OAuth compliance, logging practices, and vulnerability management.
15.6	Monitor Service Providers	Monitor service providers consistent with the enterprise's service provider management policy. Monitoring may include periodic reassessment of service provider compliance, monitoring service provider release notes, and dark web monitoring.			●	Monitor third-party MCP server providers for security-relevant changes, including version updates, capability changes, authorization behavior changes, and advisories affecting MCP frameworks and dependencies. Review provider release notes and track risks introduced by new tools, resources, or prompts.
15.7	Securely Decommission Service Providers	Securely decommission service providers. Example considerations include user and service account deactivation, termination of data flows, and secure disposal of enterprise data within service provider systems.			●	Securely decommission MCP service providers by removing approvals and allowlist entries, revoking OAuth authorizations and tokens, terminating data flows, and validating enterprise data removal from provider systems. Ensure workflows are migrated without leaving residual tool access paths.

Additional MCP Considerations

- Require clear data handling, retention, and logging expectations for third-party servers receiving enterprise data.
- Define exit and substitution plans including workflow migration and access revocation.
- Require transparency into sub-processors, registries, and third-party dependencies used to deliver MCP services.
- To mitigate registry-related risks, implement artifact mirroring for all third-party MCP components. Maintain a private, internal mirror of approved registry entries where each server's provenance has been vetted and its cryptographic hashes verified against the official publisher's records. Restrict production hosts from reaching external community registries directly to prevent accidental execution of unverified or typosquatted servers. This guidance directly addresses two CVEs documented in [Appendix B](#): CVE-2025-66580 (RCE via malicious server config) and CVE-2025-54073 (command injection in unsafe tool wrappers).
- Periodically test provider kill-switch procedures and confirm that dependent workflows fail gracefully or route to fallback paths. Document test results and use them to set realistic response-time expectations before an incident requires the procedure for real.

Control 16: Application Software Security

Manage the security life cycle of in-house developed, hosted, or acquired software to prevent, detect, and remediate security weaknesses before they can impact the enterprise.

MCP Applicability

MCP components function as applications with exposed interfaces, meaning their schemas, message handling, and authorization logic all contribute to the attack surface. Treating MCP servers, gateways, and tool wrappers as full application endpoints helps highlight where validation and control are needed to keep interactions predictable and secure. Treat all model-supplied tool parameters and context as untrusted input requiring server-side validation and safe execution.

Key application security considerations in MCP environments include:

- Weak validation of tool parameters against declared schemas, missing server-side checks, or inadequate isolation of high-impact tools can allow model-supplied inputs to trigger unsafe actions without appropriate safeguards.
- Inconsistent authorization for tools and resources can allow unauthorized access. Incorrect JSON-RPC handling can allow malformed messages to bypass controls.
- Treating tool, resource, and prompt declarations as uncontrolled interfaces can lead to unintended exposure if changes to capabilities are not reviewed, tested, and aligned to least privilege defaults.
- MCP deployments that include third-party servers or community tooling without supply chain controls, such as dependency review, integrity verification, and patch tracking, are more susceptible to introducing compromised or vulnerable components into the application security life cycle.

Safeguards

CIS Control 16: Application Software Security

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
16.1	Establish and Maintain a Secure Application Development Process	Establish and maintain a secure application development process. In the process, address such items as: secure application design standards, secure coding practices, developer training, vulnerability management, security of third-party code, and application security testing procedures. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.		●	●	Establish a secure Software Development Lifecycle (SDLC) incorporating threat modeling across tools, resources, and prompts. Apply secure coding to JSON-RPC handlers, implement OAuth correctly, protect communications with TLS, maintain accurate capability declarations, and configure privacy-preserving logging. Perform security testing before deployment.

CIS Control 16: Application Software Security

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
16.2	Establish and Maintain a Process to Accept and Address Software Vulnerabilities	Establish and maintain a process to accept and address reports of software vulnerabilities, including providing a means for external entities to report. The process is to include such items as: a vulnerability handling policy that identifies reporting process, responsible party for handling vulnerability reports, and a process for intake, assignment, remediation, and remediation testing. As part of the process, use a vulnerability tracking system that includes severity ratings and metrics for measuring timing for identification, analysis, and remediation of vulnerabilities. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard. Third-party application developers need to consider this an externally-facing policy that helps to set expectations for outside stakeholders.		●	●	Establish a repeatable process to intake, triage, remediate, verify, and close vulnerabilities across MCP servers, tool handlers, gateways, and authorization flows. Provide intake channels for internal reports, coordinated external disclosures, and upstream advisories affecting MCP SDKs and dependencies. Prioritize by exploitability and impact, focusing on tool authorization bypasses, Origin and CORS (Cross-Origin Resource Sharing) validation failures, credential exposure in logs, and JSON-RPC handling defects.
16.3	Perform Root Cause Analysis on Security Vulnerabilities	Perform root cause analysis on security vulnerabilities. When reviewing vulnerabilities, root cause analysis is the task of evaluating underlying issues that create vulnerabilities in code, and allows development teams to move beyond just fixing individual vulnerabilities as they arise.		●	●	When security vulnerabilities are identified in MCP components, perform root cause analysis to determine the underlying cause, such as missing input validation, incorrect scope enforcement, insufficient Origin checks, unsafe defaults, or dependency weaknesses. Assess whether the defect reflects a systemic gap, such as a pattern of missing server-side authorization checks or repeated reliance on client-side validation, rather than an isolated coding error. Incorporate findings into secure development standards, code review checklists, and testing requirements to prevent recurrence of similar vulnerabilities.
16.4	Establish and Manage an Inventory of Third-Party Software Components	Establish and manage an updated inventory of third-party components used in development, often referred to as a “bill of materials,” as well as components slated for future use. This inventory is to include any risks that each third-party component could pose. Evaluate the list at least monthly to identify any changes or updates to these components, and validate that the component is still supported.		●	●	Maintain an inventory of third-party MCP servers and review each component before deployment by recording its declared capabilities including tool schemas, resource URIs, and prompt definitions. As part of the inventory process, verify OAuth scope minimization, assess JSON-RPC quality, confirm logging compliance expectations, and record initialization test results to ensure only vetted components are approved for use.
16.5	Use Up-to-Date and Trusted Third-Party Software Components	Use up-to-date and trusted third-party software components. When possible, choose established and proven frameworks and libraries that provide adequate security. Acquire these components from trusted sources or evaluate the software for vulnerabilities before use.		●	●	Use current, actively maintained versions of third-party MCP components including server frameworks, client SDKs, JSON-RPC libraries, OAuth libraries, and transport dependencies. Obtain components only from trusted sources such as enterprise-approved registries or verified repositories. Monitor for upstream deprecation, maintainer abandonment, and known vulnerabilities. Replace unmaintained components before they become liabilities.
16.6	Establish and Maintain a Severity Rating System and Process for Application Vulnerabilities	Establish and maintain a severity rating system and process for application vulnerabilities that facilitates prioritizing the order in which discovered vulnerabilities are fixed. This process includes setting a minimum level of security acceptability for releasing code or applications. Severity ratings bring a systematic way of triaging vulnerabilities that improves risk management and helps ensure the most severe bugs are fixed first. Review and update the system and process annually.		●	●	Establish a vulnerability severity rating approach for MCP defects that accounts for exploitability, impact on agent logic, capability misuse potential, data-exposure risk, and cross-component propagation. Apply severity-based SLAs with accelerated response for vulnerabilities affecting OAuth flows or capability negotiation.

CIS Control 16: Application Software Security

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
16.7	Use Standard Hardening Configuration Templates for Application Infrastructure	Use standard, industry-recommended hardening configuration templates for application infrastructure components. This includes underlying servers, databases, and web servers, and applies to cloud containers, Platform as a Service (PaaS) components, and SaaS components. Do not allow in-house developed software to weaken configuration hardening.		●	●	Apply standard hardening templates when deploying MCP servers and supporting infrastructure, including baseline configuration of runtime environments, network settings, authentication mechanisms, and logging policies. Ensure these templates enforce package-integrity requirements, restrict installations to verified repositories, validate specification-version compatibility (including PKCE and Client ID Metadata Documents), and mandate hash/signature checks during provisioning.
16.8	Separate Production and Non-Production Systems	Maintain separate environments for production and non-production systems.		●	●	Maintain separate MCP environments for production and non-production use, including distinct server registries, gateway configurations, authorization server instances or scopes, and credential sets. Prevent non-production servers from accessing production data sources or authorization paths. Use staging environments to evaluate third-party servers and specification upgrades before production deployment.
16.9	Train Developers in Application Security Concepts and Secure Coding	Ensure that all software development personnel receive training in writing secure code for their specific development environment and responsibilities. Training can include general security principles and application security standard practices. Conduct training at least annually and design in a way to promote security within the development team, and build a culture of security among the developers.		●	●	Train developers building MCP servers, gateways, and tool handlers on secure coding practices specific to the protocol: OAuth 2.1 implementation including PKCE and token audience binding, safe JSON-RPC parsing and error handling, server-side authorization enforcement, confused-deputy risk patterns, and input validation for tool parameters. MCP server developers who lack this training will produce insecure authorization implementations regardless of surrounding policy controls.
16.10	Apply Secure Design Principles in Application Architectures	Apply secure design principles in application architectures. Secure design principles include the concept of least privilege and enforcing mediation to validate every operation that the user makes, promoting the concept of “never trust user input.” Examples include ensuring that explicit error checking is performed and documented for all input, including for size, data type, and acceptable ranges or formats. Secure design also means minimizing the application infrastructure attack surface, such as turning off unprotected ports and services, removing unnecessary programs and files, and renaming or removing default accounts.		●	●	Apply secure design principles to MCP architectures by performing structured threat modeling, identifying trust boundaries across model-to-tool, client-to-server, and server-to-backend integrations, and designing for strict capability minimization. Implement defense-in-depth measures, including OAuth protections, input validation, output filtering, and rate limiting, to ensure MCP components remain resilient against attacks.
16.11	Leverage Vetted Modules or Services for Application Security Components	Leverage vetted modules or services for application security components, such as identity management, encryption, auditing, and logging. Using platform features in critical security functions will reduce developers' workload and minimize the likelihood of design or implementation errors. Modern operating systems provide effective mechanisms for identification, authentication, and authorization and make those mechanisms available to applications. Use only standardized, currently accepted, and extensively reviewed encryption algorithms. Operating systems also provide mechanisms to create and maintain secure audit logs.		●	●	Use vetted API-validation libraries and schema-enforcement modules to ensure all MCP requests conform to the MCP specification and declared tool schemas. Implement content screening modules to inspect resource payloads and tool outputs for indirect prompt injection or hidden instructions embedded in external content before they are committed to the LLM context. For Streamable HTTP, rely on hardened, third-party request validation services to enforce method, path, header, and payload correctness rather than ad-hoc parsers.

CIS Control 16: Application Software Security

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
16.12	Implement Code-Level Security Checks	Apply static and dynamic analysis tools within the application life cycle to verify that secure coding practices are being followed.				Apply static analysis, dynamic analysis, and dependency scanning to MCP server and gateway codebases. Static analysis should cover JSON-RPC handler validation, OAuth flow implementation, and authorization enforcement logic; dependency scanning should identify vulnerable versions of OAuth libraries, JSON parsing libraries, and MCP SDK components before deployment. Block builds that introduce critical vulnerabilities or unapproved dependency changes. •
16.13	Conduct Application Penetration Testing	Conduct application penetration testing. For critical applications, authenticated penetration testing is better suited to finding business logic vulnerabilities than code scanning and automated security testing. Penetration testing relies on the skill of the tester to manually manipulate an application as an authenticated and unauthenticated user.				Perform structured security testing during the SDLC across MCP runtime components, tool wrappers, and transport layers, including abuse case execution and adversarial prompt testing. Verify authentication and authorization enforcement, message ID uniqueness, and logging compliance. Remediate high and critical findings before release. •
16.14	Conduct Threat Modeling	Conduct threat modeling. Threat modeling is the process of identifying and addressing application security design flaws within a design, before code is created. It is conducted through specially trained individuals who evaluate the application design and gauge security risks for each entry point and access level. The goal is to map out the application, architecture, and infrastructure in a structured way to understand its weaknesses.				Repeat threat modeling when MCP capability sets, authorization flows, server configurations, or deployment patterns change. New tool registrations, third-party server additions, protocol version updates, and gateway policy changes all expand or shift the attack surface in ways that invalidate prior assessments. Treat capability additions and authorization flow changes as mandatory threat model update triggers, not just development milestones. •

Additional MCP Considerations

- Review tool schemas and capability changes as security changes in code review and release gates.
- Test request handling for robustness including malformed messages and ensure handlers fail safely.
- Do not allow model outputs to directly trigger privileged tool actions; enforce allowlists, policy checks, and safe execution patterns as an intermediate layer between model output and execution.
- Treat capability declarations (tools, resources, prompts) as part of the API surface and require code review, testing, and approval for any change.

Control 17: Incident Response Management

Establish a program to develop and maintain an incident response capability (e.g., policies, plans, procedures, defined roles, training, and communications) to prepare, detect, and quickly respond to an attack.

MCP Applicability

MCP incidents often involve misuse of tools, unexpected capability exposure, or access to sensitive data, which can escalate quickly across components. Plans should cover rapid containment of MCP access paths and preserve evidence for understanding tool-driven actions. Containment levers (e.g., disable tools, block servers, revoke tokens, freeze registries) must be pre-approved and operationally tested to avoid delays during incidents.

Key incident response considerations in MCP environments include:

- Inability to quickly disable or block servers, revoke tokens or authorizations, rotate affected credentials, or restrict high-impact tools can allow ongoing misuse to continue unchecked.
- Missing or incomplete audit events tying identities and scopes to tool invocations, resource retrievals, capability changes, and session activity can limit responders' ability to reconstruct what actions occurred.
- Lack of planning for incidents driven by tool poisoning, indirect prompt injection through retrieved content, or malicious capability exposure can delay quarantine of servers, removal of tools, and validation of restored baselines.
- When third-party MCP servers are permitted, incident response procedures that do not include provider coordination steps or criteria for suspending external servers during active investigation can create gaps in containment and escalation.

Safeguards

CIS Control 17: Incident Response Management

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
17.1	Designate Personnel to Manage Incident Handling	Designate one key person, and at least one backup, who will manage the enterprise's incident handling process. Management personnel are responsible for the coordination and documentation of incident response and recovery efforts and can consist of employees internal to the enterprise, service providers, or a hybrid approach. If using a service provider, designate at least one person internal to the enterprise to oversee any third-party work. Review annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	Assign specific personnel or teams to manage MCP-related incident handling, with clear roles and authority to act. Ensure they are prepared to respond to MCP-specific incidents including unauthorized tool invocations, compromised servers, OAuth token theft, tool poisoning, prompt injection, credential exposure in logs, session hijacking, and DNS rebinding.
17.2	Establish and Maintain Contact Information for Reporting Security Incidents	Establish and maintain contact information for parties that need to be informed of security incidents. Contacts may include internal staff, service providers, law enforcement, cyber insurance providers, relevant government agencies, Information Sharing and Analysis Center (ISAC) partners, or other stakeholders. Verify contacts annually to ensure that information is up-to-date.	●	●	●	Maintain contacts for server maintainers, authorization server operators, LLM providers, gateway operators, specification maintainers, and library maintainers for coordinated response.

CIS Control 17: Incident Response Management

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
17.3	Establish and Maintain an Enterprise Process for Reporting Incidents	Establish and maintain a documented enterprise process for the workforce to report security incidents. The process includes reporting timeframe, personnel to report to, mechanism for reporting, and the minimum information to be reported. Ensure the process is publicly available to all of the workforce. Review annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	Establish and maintain a process for reporting MCP-related security incidents that covers internal discovery, external disclosure from upstream providers, and user-reported anomalies. Define reporting channels, required information such as affected server, tool, identity, and scope, and escalation criteria. Ensure the process integrates with the enterprise incident reporting workflow and accounts for incidents involving third-party MCP servers where provider coordination is required.
17.4	Establish and Maintain an Incident Response Process	Establish and maintain a documented incident response process that addresses roles and responsibilities, compliance requirements, and a communication plan. Review annually, or when significant enterprise changes occur that could impact this Safeguard.		●	●	Define containment procedures for unauthorized invocations including token revocation, tool disabling, and human confirmation enforcement. Specify server isolation steps: process termination for stdio servers, gateway blocking and server disablement procedures for Streamable HTTP. Include revocation of both access and refresh tokens. Document rapid response actions for tool poisoning, data exfiltration, prompt injection, DNS rebinding, and termination of compromised components.
17.5	Assign Key Roles and Responsibilities	Assign key roles and responsibilities for incident response, including staff from legal, IT, information security, facilities, public relations, human resources, incident responders, analysts, and relevant third parties. Review annually, or when significant enterprise changes occur that could impact this Safeguard.		●	●	Define MCP-specific incident response roles before incidents occur: gateway operator responsible for blocking servers and enforcing policy changes, authorization server operator responsible for revoking tokens and terminating sessions, server owner responsible for disabling or rolling back specific tool servers, registry administrator responsible for freezing or rolling back capability registrations, and provider liaison responsible for coordinating with third-party MCP server vendors. Pre-assign these responsibilities to named individuals and verify them through tabletop exercises.
17.6	Define Mechanisms for Communicating During Incident Response	Determine which primary and secondary mechanisms will be used to communicate and report during a security incident. Mechanisms can include phone calls, emails, secure chat, or notification letters. Keep in mind that certain mechanisms, such as emails, can be affected during a security incident. Review annually, or when significant enterprise changes occur that could impact this Safeguard.		●	●	No Additional MCP Guidance
17.7	Conduct Routine Incident Response Exercises	Plan and conduct routine incident response exercises and scenarios for key personnel involved in the incident response process to prepare for responding to real-world incidents. Exercises need to test communication channels, decision making, and workflows. Conduct testing on an annual basis, at a minimum.		●	●	Include MCP scenarios in tabletop exercises and ensure lessons learned feed back into registry governance, monitoring, and authorization policy. Threat scenarios may include tool poisoning response, prompt injection handling, and data exfiltration through MCP channels. Training exercises should include simulating a server compromise requiring stdio termination and Streamable HTTP session invalidation, testing token revocation workflows under time pressure, and validating that responders can identify MCP-specific indicators of compromise in logs, including anomalous tool call sequences and unexpected resource access patterns. Coordinate exercises across security operations, AI/ML teams, and MCP server owners.

CIS Control 17: Incident Response Management

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
17.8	Conduct Post-Incident Reviews	Conduct post-incident reviews. Post-incident reviews help prevent incident recurrence through identifying lessons learned and follow-up action.		●	●	Analyze the chain of events leading to the incident, including tool invocations and parameters, resource access patterns, capability changes, authorization decisions, and session activity. Determine whether the incident stemmed from malicious input such as tool poisoning or prompt injection, misconfiguration of servers or authorization policies, compromised dependencies, or gaps in monitoring and policy enforcement. Identify shortcomings in detection coverage, containment speed, and evidence availability. Integrate lessons learned into playbooks, detection rules, authorization policies, server governance processes, and SDLC release gates so that confirmed attack paths are blocked at the development and approval stage before the next capability addition or specification upgrade introduces them again.
17.9	Establish and Maintain Security Incident Thresholds	Establish and maintain security incident thresholds, including, at a minimum, differentiating between an incident and an event. Examples can include: abnormal activity, security vulnerability, security weakness, data breach, privacy incident, etc. Review annually, or when significant enterprise changes occur that could impact this Safeguard.			●	Define escalation thresholds for MCP security events including data exfiltration through tools, privilege escalation via confused deputy attacks, authorization server compromise, successful DNS rebinding, coordinated tool poisoning, and large-scale session hijacking.

Additional MCP Considerations

- Ensure responders can access and preserve evidence including tool logs, resource records, and gateway policy history.
- For third-party servers, pre-define provider coordination steps and decision points for suspending services.
- Preserve registry snapshots, capability baselines, gateway policy versions, and token and grant change logs as evidence artifacts.
- Include a registry freeze as an explicit containment lever: suspend all changes to the server registry and capability allowlists during active incidents to prevent capability expansion while an investigation is ongoing. Pre-define the authority and process for initiating and lifting a registry freeze to avoid delays during an incident.

Control 18: Penetration Testing

Test the effectiveness and resiliency of enterprise assets through identifying and exploiting weaknesses in controls (people, processes, and technology) and simulating the objectives and actions of an attacker.

MCP Applicability

Penetration testing for MCP environments should validate not only network exposure but whether tool execution paths can be manipulated for unintended actions or data access. Testing should focus on controls constraining tool use, enforcing authorization, and preventing local or cross-origin abuse.

Key penetration testing considerations in MCP environments include:

- Untested exposure to prompt injection or poisoned content can allow unsafe tool actions, data disclosure, or bypass of approval and policy controls.
- Lack of testing for OAuth failure modes, such as mis-scoped access, token replay, or authorization bypass conditions, can allow tool invocation beyond intended permissions.
- Insufficient testing of cross-origin protections and session handling for Streamable HTTP deployments, as well as localhost exposure and DNS rebinding paths, can leave local services reachable in unintended ways.

Safeguards

CIS Control 18: Penetration Testing

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
18.1	Establish and Maintain a Penetration Testing Program	Establish and maintain a penetration testing program appropriate to the size, complexity, industry, and maturity of the enterprise. Penetration testing program characteristics include scope, such as network, web application, Application Programming Interface (API), hosted services, and physical premise controls; frequency; limitations, such as acceptable hours, and excluded attack types; point of contact information; remediation, such as how findings will be routed internally; and retrospective requirements.		●	●	Include MCP environments in the enterprise penetration testing program with explicitly defined scope: gateways, Streamable HTTP servers, authorization flows, stdio endpoints, high-impact tool execution paths, and server registries. MCP-specific attack surfaces including OAuth implementation, capability governance, JSON-RPC parsing robustness, and stdio process boundaries are not covered by standard application or network penetration testing scopes and must be named explicitly to ensure coverage.
18.2	Perform Periodic External Penetration Tests	Perform periodic external penetration tests based on program requirements, no less than annually. External penetration testing must include enterprise and environmental reconnaissance to detect exploitable information. Penetration testing requires specialized skills and experience and must be conducted through a qualified party. The testing may be clear box or opaque box.		●	●	Test all internet-accessible MCP servers, gateways, and authorization servers, assessing OAuth vulnerabilities such as PKCE bypass, scope escalation, and token theft. Evaluate authorization bypass paths, DNS rebinding, session hijacking, CORS misconfigurations, JSON-RPC parsing robustness, request and response correlation handling, and safe error behavior.
18.3	Remediate Penetration Test Findings	Remediate penetration test findings based on the enterprise's documented vulnerability remediation process. This should include determining a timeline and level of effort based on the impact and prioritization of each identified finding.		●	●	Address weak OAuth configurations, missing human-in-the-loop (HITL) confirmations, inadequate tool parameter validation, and logging deficiencies. Remediate authorization bypass paths, tool poisoning vectors, and prompt injection risks. Correct DNS rebinding, session fixation, and JSON-RPC compliance issues.

CIS Control 18: Penetration Testing

Safeguard	Title	Description	IG1	IG2	IG3	MCP Applicability
18.4	Validate Security Measures	Validate security measures after each penetration test. If deemed necessary, modify rulesets and capabilities to detect the techniques used during testing.				After MCP penetration tests identify exploitable paths, validate that remediations are effective through retesting before closing findings. Validation should confirm that gateway policy updates, capability allowlist changes, and OAuth implementation fixes close the specific paths exploited, and that regression test cases have been added to prevent reintroduction through future spec upgrades, SDK updates, or capability changes. •
18.5	Perform Periodic Internal Penetration Tests	Perform periodic internal penetration tests based on program requirements, no less than annually. The testing may be clear box or opaque box.				Test stdio servers for privilege escalation, filesystem access beyond declared roots, and environment variable exposure. Test internal Streamable HTTP servers for authorization bypass. Assess gateways for policy bypass and session weaknesses. Attempt tool invocation without required scopes, bypass human confirmation, access resources outside approved URIs, and test for confused deputy behavior. Review logs for credential exposure. •

Additional MCP Considerations

- Include tests attempting to manipulate tool use through hostile content; verify approval, policy, and validation controls prevent unsafe actions.
- Test authorization boundaries through mis-scoped access, token replay, and privilege escalation.
- Test transport and local exposure paths applicable to deployment patterns.
- Test for tool name shadowing where attackers register tools with common names to masquerade as legitimate functionality.
- Test capability drift scenarios (new tools, resources, prompts appearing) and validate deny-by-default behavior until re-approval.

Conclusion

MCP provides a structured, predictable way to manage how AI systems interact with tools, data, and external services. By treating MCP servers, tool interfaces, datasets, and model endpoints as managed enterprise assets, enterprises can apply familiar Safeguards, such as access control, secure configuration, and continuous monitoring, to a new class of AI components. MCP does not replace existing security practices; instead, it offers a clear, enforceable boundary where those practices can be applied consistently to AI workflows. As enterprises adopt AI at scale, aligning MCP implementations with the CIS Controls ensures that the operational benefits of standardized AI integration are matched with strong, measurable security outcomes.

Appendix A: MCP Deployment Security Patterns

This appendix provides security guidance for each MCP deployment pattern. Individual Control sections reference this appendix for shared considerations and include only guidance unique to that control domain.

This appendix also includes security guidance for two official MCP extensions: MCP Applications (A.5) and MCP Authorization Extensions (A.6), which introduce additional security considerations beyond the core deployment patterns.

The four deployment patterns are:

- **Local stdio (subprocess):** MCP servers run as subprocesses spawned by the host, communicating over ``stdin`` and ``stdout`` under the local user context.
- **Remote Streamable HTTP (network):** MCP servers are accessed over the network using a single HTTP endpoint, with optional Server-Sent Events for server-to-client streaming.
- **Gateway-Mediated (central enforcement):** MCP traffic is routed through a centralized gateway that enforces policy, aggregates logging, and manages access across multiple backend servers.
- **Third-Party Servers (external supply chain):** Third-party MCP servers sourced from registries or external providers require additional governance.

A.1: Local stdio Server Security

stdio MCP servers run as subprocesses spawned by the MCP host application. This architecture creates specific security characteristics that inform control implementation.

In the local stdio pattern, the host application starts the MCP server as a local subprocess and communicates via standard input and output. All components run on the same endpoint under the local user context, with no network exposure between the host and MCP server. The diagram below illustrates the component relationships and primary security boundaries in this deployment pattern.

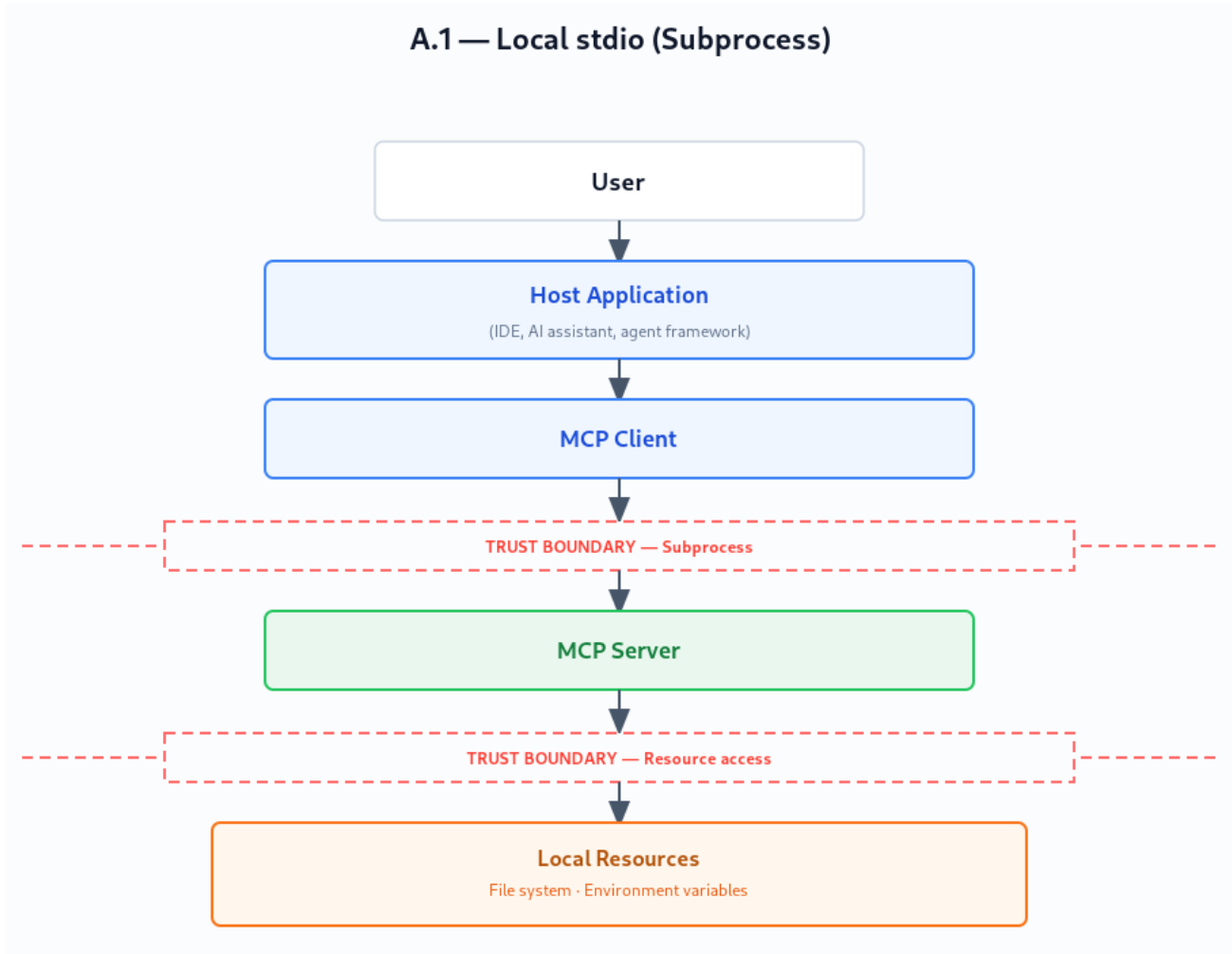


Figure 1: A.1 – Local stdio (Subprocess)

Trust Boundaries and Privileges

Since there is no network-layer authentication in the trust boundary between the client and server in the stdio pattern, security relies on process isolation, OS-level permissions, and the integrity of the installed server executable. This primary security boundary is between the host application and the stdio subprocess. The MCP server communicates with the host application only through ``stdin`` and ``stdout``. ``stderr`` is for diagnostics and does not affect protocol behavior.

The stdio server runs with the privileges of the host user. It inherits access to local files, environment variables, network resources, and any credentials available to that user. Without OS controls or sandboxing, it can reach any resource available to that user context.

For AI agent deployments, this privilege scope is often broader than required. Constrain agents to the minimum privileges needed for their tasks. Use OS sandboxing, dedicated low-privilege service accounts, or container isolation to limit access to only the required files, networks, and credentials.

The trust boundary between an MCP server and the resources it accesses depends on the nature of the resource. Where the server accesses local resources (e.g., files on the local file system, data in the execution environment), the security model is the same as the client-server trust boundary described above. Where the server accesses remote resources over the network, the security model should follow the guidance in Controls 5, 6, and 12, as well as any other relevant Controls.

Transport and Session Security

Messages use JSON-RPC and are exchanged as newline-delimited JSON over ``stdin`` and ``stdout``. Servers must not write non-MCP content to ``stdout`` as this corrupts the protocol stream. Request IDs must be unique within a session and must not be reused. Notifications, which are one-way messages expecting no response, omit the ID field entirely.

stdio deployments do not use network-layer authentication. Security rests on local process boundaries, the OS user and groups the process runs under, and file system permissions.

Where a stdio server makes outbound network connections to remote services, those connections are subject to the same transport security requirements as Streamable HTTP deployments; see Controls 3.10, 5, 6, and 12.

Configuration and Life Cycle

Server executables must be verified before launch through hash validation or signature checking. Store server configurations in protected locations with appropriate file system permissions. Environment variables passed to servers may contain sensitive values and should be minimized to required items only. Logging output to ``stderr`` must not contain credentials, secrets, or PII. Where the host stores tokens locally, use OS-provided secure storage (e.g., system keychains) and avoid plaintext files or long-lived caches.

Monitoring and Containment

The host controls the server life cycle through process signals, including ``SIGTERM`` for graceful shutdown and ``SIGKILL`` for forced termination. Implement timeout policies for server responses; unresponsive servers should receive ``SIGTERM`` followed by ``SIGKILL`` after a defined interval. MCP server crash recovery should include capability re-negotiation through a fresh initialization handshake.

A.2: Remote Streamable HTTP Server Security

Streamable HTTP servers expose MCP capabilities over network connections using a single HTTP endpoint with optional Server-Sent Events for server-to-client streaming. This introduces network-layer trust boundaries that do not exist in stdio deployments.

In this pattern, the MCP client connects directly to a remote MCP server over HTTPS using Streamable HTTP transport. The MCP server is responsible for validating OAuth 2.1 tokens from the MCP client, enforcing access controls, and managing its own connection to backend systems. The diagram below illustrates the component relationships and network trust boundary in this deployment pattern.

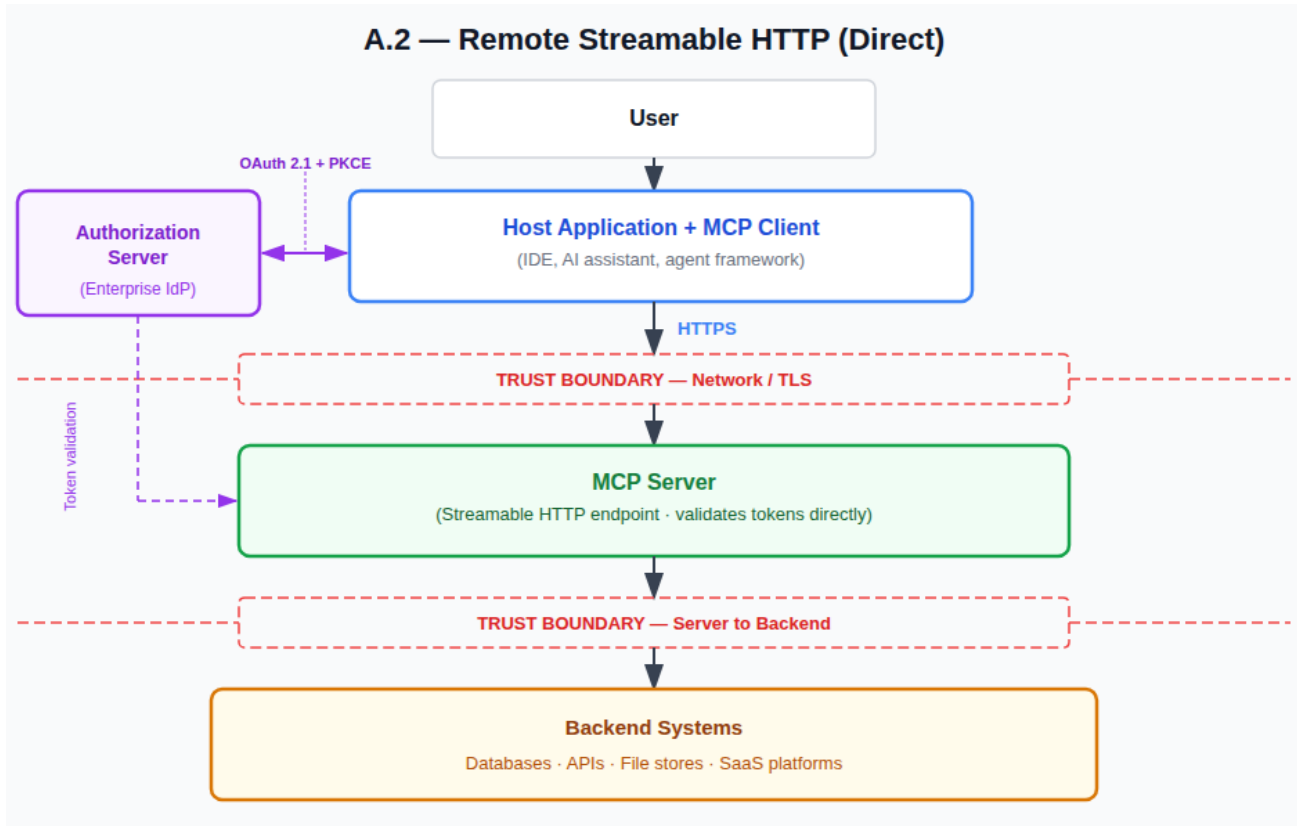


Figure 2: A.2 – Remote Streamable HTTP (Direct)

Trust Boundaries and Privileges

The trust boundary shifts from a local process boundary to a network boundary between the MCP client and the MCP server. Servers may be exposed to the internet or accessed across network segments, making authentication, encryption, and Origin enforcement essential. Unlike stdio MCP servers, Streamable HTTP servers must independently validate every request since they cannot rely on OS-level process isolation to establish trust.

Transport and Session Security

TLS 1.2 or better is required for all connections. MCP servers must validate client certificates when mutual TLS is deployed. MCP servers intended for local access only must bind to `127.0.0.1` rather than 0.0.0.0.

Enforce Origin validation using an explicit allowlist of expected Origins and reject missing, unexpected, or null Origin values. Apply the same validation to both standard HTTP requests and any server-to-client streaming connections (e.g., streaming responses), and do not rely on CORS configuration as a substitute for server-side Origin enforcement. Origin validation reduces DNS rebinding and cross-request risks.

Sessions are identified through the `MCP-Session-Id` header. Servers must validate this identifier on all requests after initialization. Session termination occurs through `HTTP DELETE` requests containing the session identifier. Implement session timeout policies aligned with organizational security requirements.

Configuration and Life Cycle

Streamable HTTP uses a single HTTP endpoint for all protocol operations. Implement rate limiting per tool per client identity. Monitor for unusual request patterns indicating automated abuse. Protect against resource exhaustion through request size limits and connection pooling.

MCP servers declare capabilities during the initialization handshake. Declared capabilities must accurately reflect available features. The `listChanged` notification mechanism enables dynamic capability updates when tool, resource, or prompt availability changes.

Monitoring and Containment

Monitor for connection rate spikes, traffic to unauthorized destinations, session anomalies, and unencrypted traffic where TLS is required. Streamable HTTP deployments typically generate higher event volume than stdio due to request and response activity, batching, and session tracking. Containment actions include session termination via `HTTP DELETE`, token revocation, and gateway-level traffic blocking.

A.3: Gateway-Mediated Deployment Security

MCP gateways provide centralized policy enforcement, logging aggregation, and access control across multiple backend servers. Gateways function as an intermediary control point and are critical infrastructure in production deployments.

In this pattern, an MCP gateway sits between the MCP client and one or more backend MCP servers, acting as a centralized policy enforcement point. The gateway intercepts all MCP traffic, performing token validation, Origin checking, rate limiting, and audit logging before requests reach the backend. This architectural interposition makes the gateway both the primary security control surface and the single point of visibility across the deployment. The diagram below shows the key components and trust relationships in this pattern.

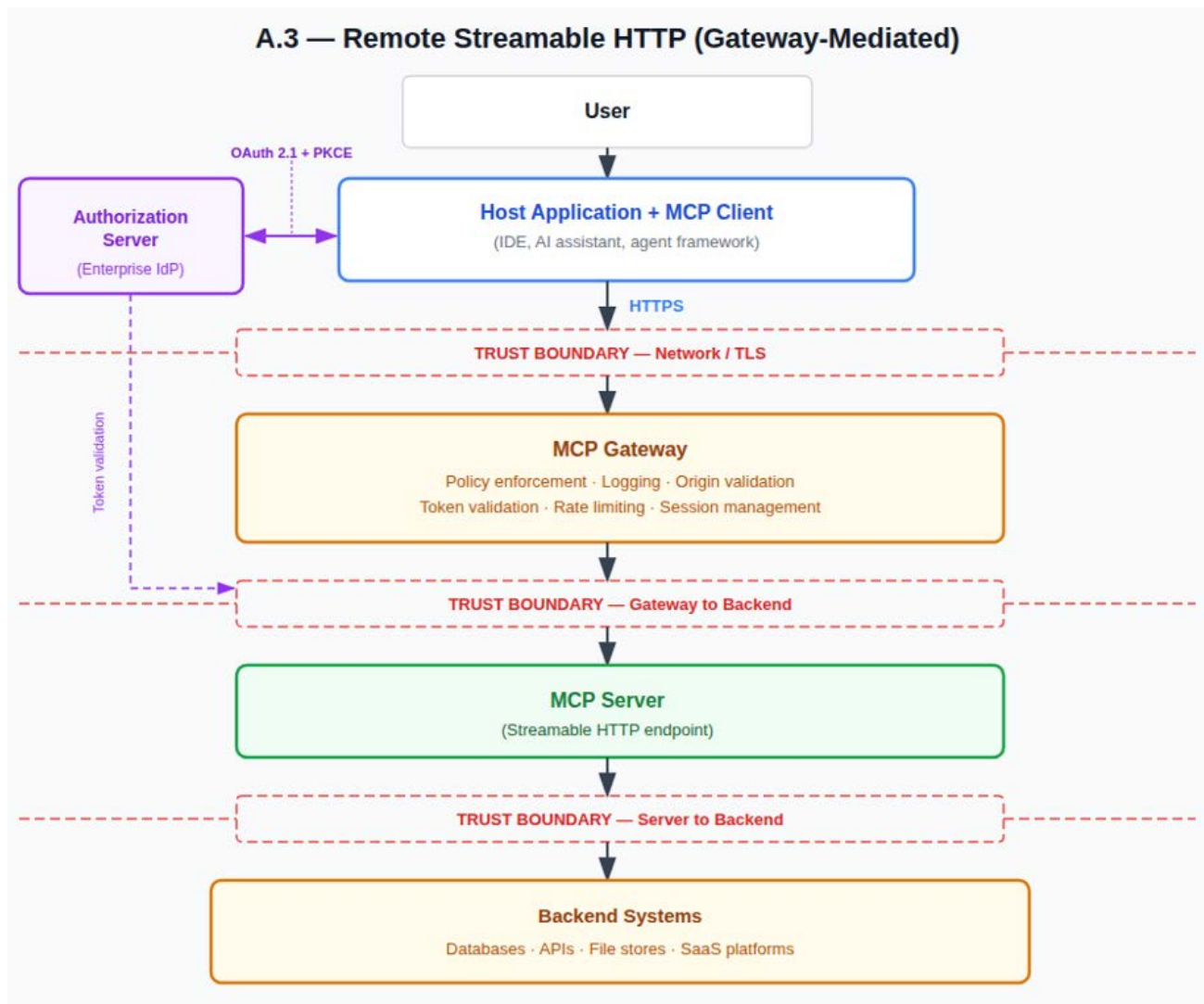


Figure 3: A.3 – Remote Streamable HTTP (Gateway-Mediated)

Trust Boundaries and Privileges

Gateways may terminate OAuth authentication and establish separate trust relationships with backend servers. This architecture enables credential isolation where backend servers never receive end-user tokens. The gateway becomes the enforcement boundary for authorization, scope validation, and policy decisions.

Implement credential management for gateway-to-server authentication with appropriate rotation policies. Ensure that the gateway's own administrative interfaces are protected with strong authentication and restricted to authorized personnel.

Transport and Session Security

Gateways inherit the transport security requirements of Streamable HTTP (TLS 1.2+, Origin validation, session management). In addition, gateways must secure the connection between the gateway and each backend server. Where backend servers are co-located or on a trusted network, internal TLS or mutual TLS provides defense in depth.

Session state managed by the gateway must be preserved consistently across failover events to avoid session loss or policy gaps during infrastructure transitions.

Configuration and Life Cycle

Gateways enforce tool invocation policies including allowlists, rate limits, and required approvals. Implement rate limits at the tool level per user or per identity to prevent individual agents or sessions from exhausting backend API quotas or overwhelming downstream services. Policy decisions should occur before requests reach backend servers. Implement consistent policy across all mediated servers rather than per-server configurations where possible.

Gateway infrastructure requires high availability given its role as a control point. Implement failover mechanisms that preserve session state and policy consistency. Monitor gateway health and performance as critical infrastructure.

Monitoring and Containment

Aggregate logging from all backend servers through the gateway. Correlate requests across the gateway and backend for complete audit trails. Ensure log aggregation does not create single points of failure for forensic data. Gateway logs must not contain credentials, secrets, or PII.

Gateways enable rapid response to security incidents through centralized server disconnection, tool disabling, and traffic blocking without requiring changes to individual backend servers.

A.4: Third-Party MCP Server Security

Most enterprise MCP deployments will rely heavily on third-party servers sourced from community registries, open-source repositories, or commercial providers. These servers require additional governance controls beyond enterprise-developed servers because they introduce supply chain risk, reduced visibility into implementation details, and dependency on external maintainers.

Trust Boundaries and Privileges

Limit third-party server access to required resources through gateway-mediated deployments where possible. Implement additional logging and alerting for third-party server activity. Consider network segmentation separating third-party server traffic from internal resources.

Third-party servers may run locally via stdio or remotely via Streamable HTTP. The trust boundary concerns of the underlying transport (A.1 or A.2) apply in addition to the governance controls in this section.

Pre-Deployment Assessment

Evaluate MCP servers before deployment through capability review, code audit when source is available, and dynamic analysis when it is not. Verify that declared capabilities match documented behavior. Assess maintainer reputation, commit history, and security disclosure practices.

Supply Chain Verification

Track MCP server software provenance including repository, build pipeline, and dependency sources. Verify package integrity through published hashes and signatures. Monitor for typosquatting attacks using names similar to legitimate packages. Establish a trust anchor for discovery by preferring enterprise-approved registries and signed metadata, and refuse connections to servers that are not allowlisted or that fail integrity verification.

Contractual Controls

Establish contractual security requirements when third parties operate servers or their downstream integrations. Define data handling requirements including residency, retention, and breach notification. Require security certifications or audit evidence appropriate to data sensitivity.

Ongoing Monitoring

Monitor third-party MCP servers for behavioral changes, unexpected capability modifications, and anomalous access patterns. Establish processes for responding to upstream security advisories. Maintain awareness of MCP server maintenance status and plan for deprecation or abandonment scenarios.

A.5: MCP Apps Extension Security

Some MCP deployments may support the MCP Apps extension, which allows an MCP server to provide interactive user interface content rendered by the host. Unlike tools and resources that primarily return data over MCP, MCP Apps introduce web-rendered content into the workflow and therefore expand the attack surface, particularly for Streamable HTTP and third-party servers.

Threat Model Implications

MCP Apps add a browser execution context to the trust boundary. Malicious or compromised MCP app content can attempt to mislead users, influence decisions, or trigger unintended actions. MCP app content should be treated as untrusted, especially when sourced from third-party servers or registries.

Isolation and Content Restrictions

Hosts should render MCP app content with strong isolation controls (e.g., sandboxed iframe rendering) and enforce restrictive content policies that limit script execution and external network access. Restrict allowed Origins and minimize the set of destinations the MCP app is permitted to load. Do not allow MCP app content to directly access host credentials, tokens, or sensitive local resources.

Permissions and User Awareness

If the host supports MCP app-requested permissions (e.g., microphone, camera, clipboard, or file selection), default to deny and require explicit user approval. Ensure the user can view the requested permissions, which server requested them, and what data will be accessed or transmitted.

Governance and Monitoring

Apply allowlisting and change control for servers that provide MCP Apps and for any MCP app user interface resources they deliver. Log MCP app loading events, permission grants, and user-visible actions to support audit and incident response. Treat MCP app updates as security-relevant changes and review them with the same rigor as tool or schema changes.

A.6: MCP Authorization Extension Security

MCP authorization extensions provide supplementary authentication mechanisms beyond the core OAuth 2.1 authorization code flow. Two official extensions are published under the `io.modelcontextprotocol` namespace: OAuth Client Credentials and Enterprise-Managed Authorization. Extensions are disabled by default and require explicit opt-in; only enable extensions that are actively needed and supported by both client and server. Verify that extension capabilities exchanged during initialization match expected values and reject sessions where unexpected extensions are advertised.

OAuth Client Credentials

Enables machine-to-machine authorization using the OAuth 2.0 Client Credentials grant, applicable when no human user is present (e.g., in automated pipelines or server-to-server integrations). Store client secrets in an enterprise vault or secrets manager with audited access and enforce rotation policies. Client credential tokens do not carry user identity; confirm downstream systems apply appropriate controls when receiving machine-identity tokens. Audit client credential token issuances separately from user-delegated tokens and monitor for scope escalation or anomalous usage patterns.

Enterprise-Managed Authorization

Places the enterprise Identity Provider (IdP) in the authorization path, enabling corporate SSO to govern which users may access which MCP servers and with which scopes. Confirm the IdP enforces MCP-specific access policies and treat IdP policy configuration as a security-sensitive change requiring change management review. Validate that servers reject tokens with incorrect audience claims and that clients do not cache or reuse short-lived authorization grants beyond their intended lifetime. Monitor IdP token exchange events for anomalous patterns such as a single client requesting grants for an unusual number of servers.

Governance and Monitoring

Apply allowlisting and change control to which authorization extensions are permitted in production. Log extension negotiation during session initialization and alert on sessions using unapproved extensions.

Appendix B: MCP CVEs Mapped to Best Practices

The following table maps known MCP-related Common Vulnerabilities and Exposures (CVEs) to the Safeguards in this guide. These entries demonstrate that the threats addressed by this guide are grounded in real-world vulnerabilities disclosed in MCP implementations and related components, not theoretical risks. Enterprises can use this mapping to validate that their control implementations address the specific weakness classes that have already been exploited or disclosed in the MCP ecosystem. As the MCP ecosystem matures, new CVEs may emerge that map to additional Safeguards.

CVE	Affected Component	Vulnerability Theme	MCP Focus	Most Effective Safeguards
CVE-2025-47274	ToolHive	Secrets exposure in run configuration files	General MCP security control	3.11; 3.14; 4.1; 5.5
CVE-2025-49596	MCP Inspector	Remote code execution in inspection/testing tool	Code execution / unsafe tool wrapper	2.5; 4.1; 12.2; 16.7
CVE-2025-52573	ios-simulator-mcp	Command injection via exec	Code execution / unsafe tool wrapper	2.5; 2.7; 12.2; 16.2
CVE-2025-53110	MCP Filesystem server	Path/prefix validation bypass (file access escape)	General MCP security control	2.5; 3.3; 4.1; 18.5
CVE-2025-53365	MCP Python SDK	Denial of Service (DoS) / crash in Streamable HTTP handling	Availability / resilience	7.1; 7.2; 7.5; 16.2
CVE-2025-53366	MCP Python SDK	DoS / crash in Streamable HTTP handling	Availability / resilience	7.1; 7.2; 7.5; 16.2
CVE-2025-53372	node-code-sandbox-mcp	Command injection / sandbox escape risk	Code execution / unsafe tool wrapper	2.5; 12.2; 12.8; 16.2
CVE-2025-53818	GitHub Kanban MCP Server	Command injection via gh exec	Code execution / unsafe tool wrapper	2.5; 2.7; 12.2; 16.2
CVE-2025-53832	Lara Translate MCP Server	Command injection via exec	Code execution / unsafe tool wrapper	2.5; 2.7; 12.2; 16.2
CVE-2025-54073	mcp-package-docs	Command injection	Code execution / unsafe tool wrapper	2.5; 2.7; 12.2; 16.2
CVE-2025-58358	Markdownify MCP server	Command injection via exec	Code execution / unsafe tool wrapper	2.5; 2.7; 12.2; 16.2
CVE-2025-64109	Cursor CLI	RCE via malicious .cursor/mcp.json in repo	Code execution / unsafe tool wrapper	2.5; 2.7; 14.2; 16.7
CVE-2025-6514	mcp-remote	RCE via malicious MCP server	Code execution / unsafe tool wrapper	2.5; 6.7; 12.2; 16.7
CVE-2025-6515	MCP prompt/session hijacking pattern	Prompt hijack / unsafe tool authorization pattern	Session integrity	6.1; 6.8; 8.5; 14.2
CVE-2025-66401	MCP Watch scanner	Command injection in repo clone	Code execution / unsafe tool wrapper	2.5; 2.7; 7.5; 16.2
CVE-2025-66414	MCP TypeScript SDK	DNS rebinding risk (protection not enabled by default)	Transport hardening (Origin / rebinding)	4.1; 9.2; 12.2; 18.2
CVE-2025-66416	MCP Python SDK	DNS rebinding risk (protection not enabled by default)	Transport hardening (Origin / rebinding)	4.1; 9.2; 12.2; 18.2
CVE-2025-66580	MCP server config injection	RCE via malicious server config on click	Code execution / unsafe tool wrapper	2.5; 4.6; 14.2; 16.7
CVE-2025-68143	mcp-server-git	Arbitrary path/repo init (unsafe filesystem operations)	Filesystem boundary enforcement	2.5; 3.3; 4.1; 16.2
CVE-2025-68144	mcp-server-git	Arbitrary file write via symlink (repo crafting)	General MCP security control	3.3; 4.1; 16.2; 18.5
CVE-2025-68145	mcp-server-git	Arbitrary file read via symlink (repo crafting)	General MCP security control	3.3; 4.1; 16.2; 18.5

Appendix C: CIS Controls

The CIS Critical Security Controls® (CIS Controls®) are a prioritized set of actions which collectively form a defense-in-depth set of best practices that mitigate the most common attacks against systems and networks. They are developed by a community of information technology (IT) experts who apply their first-hand experience as cyber defenders to create these globally accepted security best practices. The experts who develop the CIS Controls come from a wide range of sectors, including retail, manufacturing, healthcare, education, government, defense, and others. It is important to note that while the CIS Controls address general best practices that enterprises should implement to protect their environment, some operational environments may present unique requirements not addressed by the CIS Controls or require deviations from best practices.

Implementation Groups

The Implementation Group methodology was developed as a new way to prioritize the CIS Controls. These IGs provide a simple and accessible way to help enterprises of different classes focus their scarce security resources, while still leveraging the value of the CIS Controls program, community, and complementary tools and working aids. More about the Implementation Groups can be found in our [Guide to Implementation Groups \(IG\): CIS Critical Security Controls v8.1](#).

IG1

An IG1 enterprise is small to medium-sized with limited IT and cybersecurity expertise to dedicate toward protecting IT assets and personnel. The principal concern of these enterprises is to keep the business operational, as they have a limited tolerance for downtime. The sensitivity of the data that they are trying to protect is low and principally surrounds employee and financial information. Safeguards selected for IG1 should be implementable with limited cybersecurity expertise and aimed to thwart general, non-targeted attacks. These Safeguards will also typically be designed to work in conjunction with small or home office commercial off-the-shelf (COTS) hardware and software.

IG2

An IG2 enterprise employs individuals responsible for managing and protecting IT infrastructure. These enterprises support multiple departments with differing risk profiles based on job function and mission. Small enterprise units may have regulatory compliance burdens. IG2 enterprises often store and process sensitive client or enterprise information and can withstand short interruptions of service. A major concern is loss of public confidence if a breach occurs. Safeguards selected for IG2 help security teams cope with increased operational complexity. Some Safeguards will depend on enterprise-grade technology and specialized expertise to properly install and configure.

IG3

An IG3 enterprise employs security experts that specialize in the different facets of cybersecurity (e.g., risk management, penetration testing, application security). IG3 assets and data contain sensitive information or functions that are subject to regulatory and compliance oversight. An IG3 enterprise must address availability of services and the confidentiality and integrity of sensitive data. Successful attacks can cause significant harm to the public welfare. Safeguards selected for IG3 must abate targeted attacks from a sophisticated adversary and reduce the impact of zero-day attacks.

If you would like to know more about how the CIS Controls and Implementation Groups pertain to enterprises of all sizes, visit our website at <https://www.cisecurity.org/controls/cis-controls-list/>.

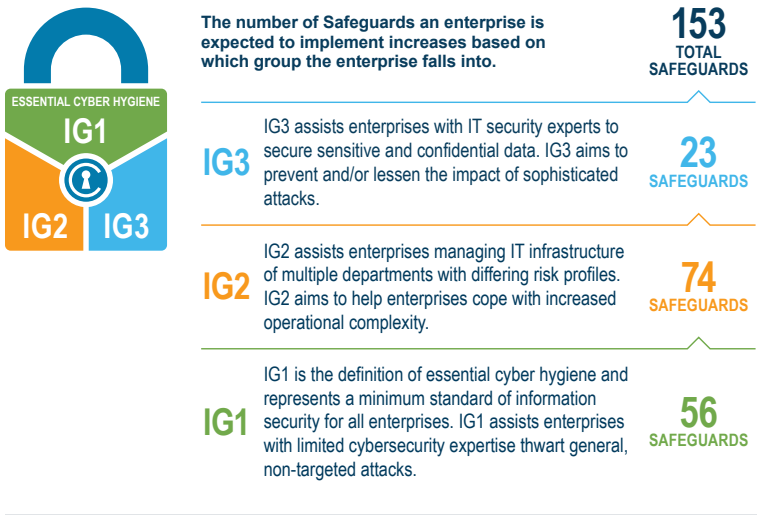


Figure 4: CIS Controls v8.1 Implementation Group levels

Appendix D: Acronyms and Abbreviations

AAA	Authentication, Authorization, and Auditing
API	Application Programming Interface
BEC	Business Email Compromise
CDM	Community Defense Model
CI	Continuous Integration
CIMD	Client ID Metadata Document
CORS	Cross-Origin Resource Sharing
COTS	Commercial Off-the-Shelf
CVE	Common Vulnerabilities and Exposures
DLP	Data Loss Prevention
DNS	Domain Name System
DoS	Denial of Service
DPoP	Demonstration of Proof-of-Possession
EDR	Endpoint Detection and Response
HITL	Human-in-the-Loop
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IaC	Infrastructure-as-Code
IdP	Identity Provider
IDS	Intrusion Detection System
IG	Implementation Group
IPS	Intrusion Prevention System
JSON	JavaScript Object Notation
JSON RPC	JSON Remote Procedure Call
LLM	Large Language Model

MCP	Model Context Protocol
MFA	Multi-Factor Authentication
OAuth	Open Authorization
PAM	Privileged Account Management
PII	Personally Identifiable Information
PKCE	Proof Key for Code Exchange
RTO	Recovery Time Objective
SBOM	Software Bill of Materials
SDK	Software Development Kit
SDLC	Software Development Lifecycle
seccomp	Secure Computing Mode
SIEM	Security Information and Event Management
SLA	Service Level Agreement
SSE	Server-Sent Events
SSO	Single Sign-On
stderr	Standard Error
stdio	Standard Input/Output
stdout	Standard Output
TLS	Transport Layer Security
TTL	Time-to-Live
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
VPN	Virtual Private Network
WAF	Web Application Firewall
ZTNA	Zero Trust Network Access

Appendix E: Links and Resources

- **CIS Critical Security Controls (CIS Controls) v8.1:** Learn more about the CIS Controls, including how to get started, why each Control is critical, procedures and tools to use during implementation, and a complete listing of Safeguards for each Control.
- **CIS Controls Policy Templates:** Policy templates geared toward Safeguards found in IG1 of the CIS Controls.
- **A Roadmap to the CIS Controls:** There is a broader ecosystem that surrounds the CIS Controls that offers guidance, tools, resources, mappings, and more to help facilitate the adoption and implementation of the framework. This guide will help adopters understand what is available to them, where to start, and how to put it all together.
- **Establishing Essential Cyber Hygiene:** IG1 is essential cyber hygiene and represents a minimum standard of information security for all enterprises. This guide will help enterprises establish essential cyber hygiene.
- **Guide to Asset Classes:** In v8.1, CIS restructured Asset Classes and their respective definitions to ensure consistency throughout the Controls. Learn more about our naming conventions and what they mean.
- **Guide to Implementation Groups (IG):** IGs are the recommended guidance to prioritize implementation of the CIS Controls. In an effort to assist enterprises of every size, IGs are divided into three groups. Learn more about the five factors that can impact IG selection for an enterprise.
- **CIS Controls Assessment Specification:** Provides an understanding of what should be measured in order to verify that the Safeguards are properly implemented.
- **CIS Controls Navigator:** Learn more about the Controls and Safeguards and see how they map to other security standards (e.g., CMMC, NIST SP 800-53 Rev. 5, PCI DSS, MITRE ATT&CK). Available for CIS Controls versions 8.1, 8, and 7.1.
- **CIS Community Defense Model (CDM) v2.0:** A guide published by CIS that leverages the open availability of comprehensive summaries of attacks and security incidents, and the industry-endorsed ecosystem that is developing around the MITRE ATT&CK Framework.
- **CIS Risk Assessment Method (CIS RAM) v2.2:** An information security risk assessment method that helps enterprises implement and assess their security posture against the CIS Controls.
- **CIS SecureSuite® Membership:** Membership with access to CIS-CAT Pro Assessor, CIS Build Kits, CIS Benchmarks, and more.
- **CIS Benchmarks®:** Secure configuration guidelines for 100+ technologies, including operating systems, applications, and network devices.
- **CIS SecureSuite Platform:** A unified platform for CIS SecureSuite Members that provides organizations with the ability to assess their cybersecurity posture against the CIS Critical Security Controls® (CIS Controls®) and to demonstrate conformance with the CIS Benchmarks®.
- **CIS Build Kits:** ZIP files that contain a Group Policy Object (GPO) for each profile within the corresponding CIS Benchmark.
- **CIS Hardened Images®:** Virtual machine images securely pre-configured to the CIS Benchmarks.
- **CIS WorkBench:** Get involved in one of our many communities.
- **CIS Password Policy Guide:** CIS guidance for secure usage of passwords in an enterprise.

Official Documentation

- **Model Context Protocol Specification:** <https://modelcontextprotocol.io/specification/2025-11-25>
- **Model Context Protocol Documentation:** <https://modelcontextprotocol.io/docs>
- **MCP GitHub Repository:** <https://github.com/modelcontextprotocol>

Specification Evolution

- **MCP Specification 2024-11-05 (Initial Release):** <https://modelcontextprotocol.io/specification/2024-11-05>
- **MCP Specification 2025-03-26:** <https://modelcontextprotocol.io/specification/2025-03-26>
- **MCP Specification 2025-11-25 (Current):** <https://modelcontextprotocol.io/specification/2025-11-25>

The Center for Internet Security, Inc. (CIS®) makes the connected world a safer place for people, businesses, and governments through our core competencies of collaboration and innovation. We are a community-driven nonprofit, responsible for the CIS Critical Security Controls® and CIS Benchmarks®, globally recognized best practices for securing IT systems and data. We lead a global community of IT professionals to continuously evolve these standards and provide products and services to proactively safeguard against emerging threats. Our CIS Hardened Images® provide secure, on-demand, scalable computing environments in the cloud. CIS is home to the Multi-State Information Sharing and Analysis Center® (MS-ISAC®), the trusted resource for cyber threat prevention, protection, response, and recovery for U.S. State, Local, Tribal, and Territorial government entities, and the Elections Infrastructure Information Sharing and Analysis Center® (EI-ISAC®), which supports the rapidly changing cybersecurity needs of U.S. election offices. To learn more, visit <http://cisecurity.org> or follow us on X: [@CISecurity](https://twitter.com/CISecurity).

-  cisecurity.org
-  email@cisecurity.org
-  518-266-3460
-  Center for Internet Security
-  @CISecurity
-  CenterforIntSec
-  cisecurity
-  TheCISecurity

